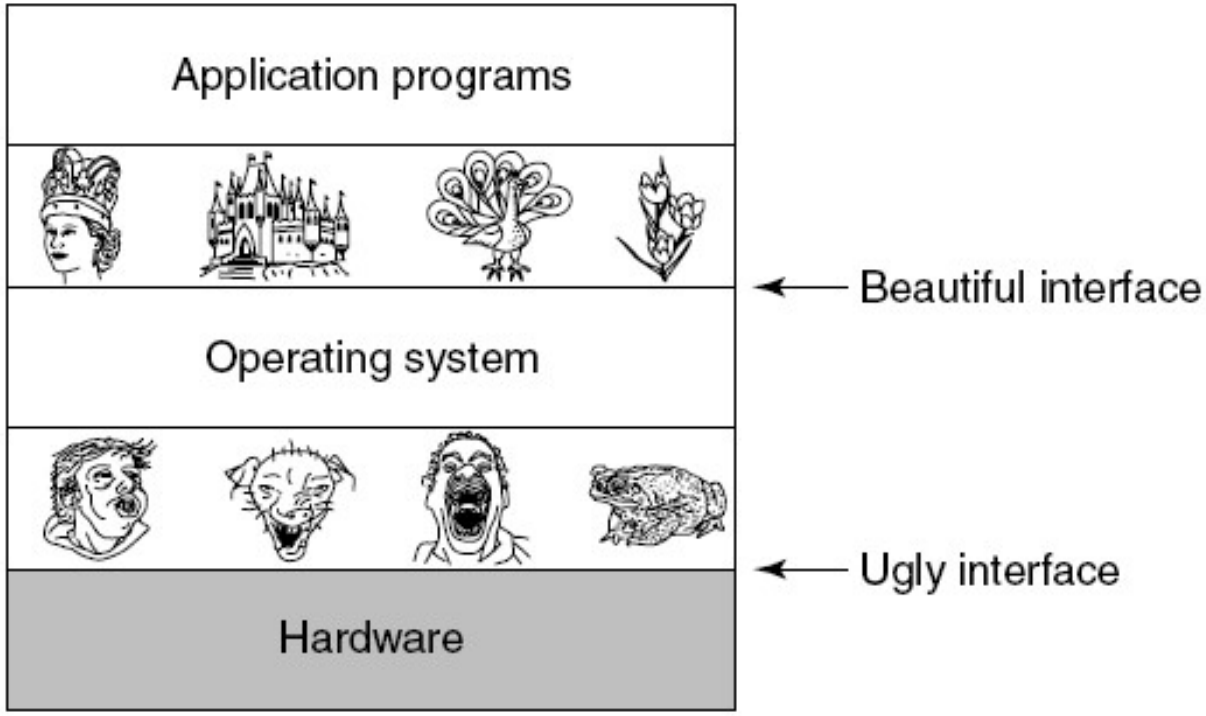
Operating System Overview

Operating System

- A program that controls the execution of application programs
- An interface between applications and hardware



Operating Systems Objectives

- Convenience
  - Makes the computer more convenient to use
- Efficiency
  - Allows computer system resources to be used in an efficient manner
- Ability to evolve
  - Permit effective development, testing, and introduction of new system functions without interfering with service

Layers and Views

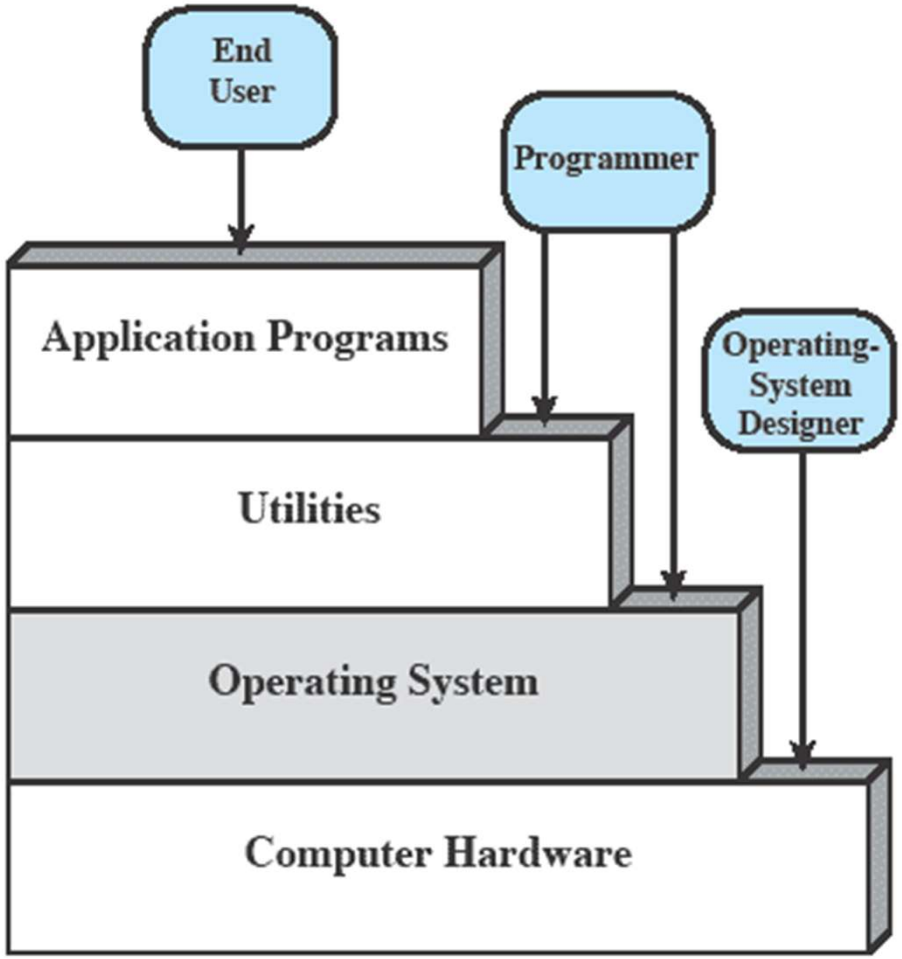


Figure 2.1 Layers and Views of a Computer System

Computer Hardware & Software Infrastructure

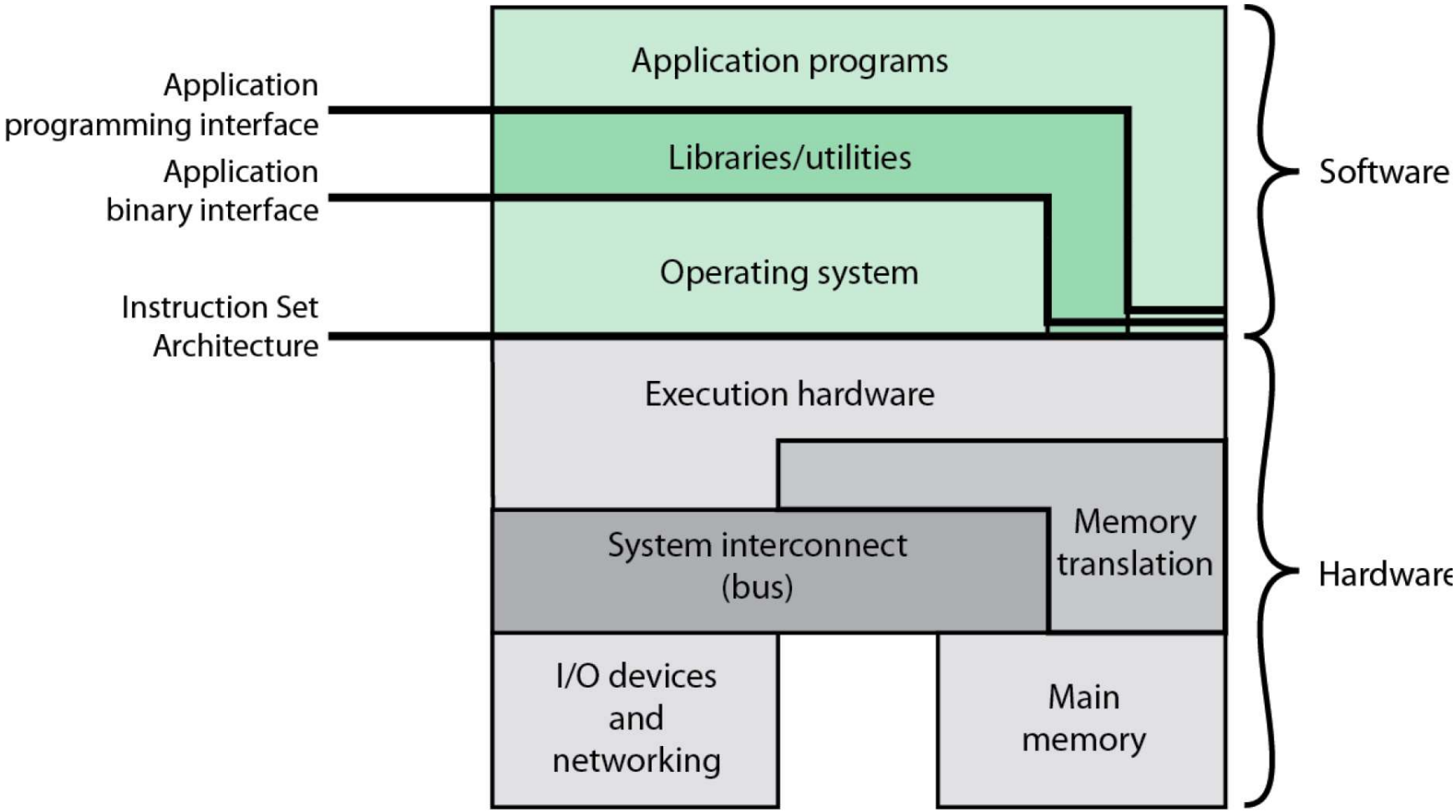


Figure 2.1 Computer Hardware and Software Infrastructure

Operating System Services

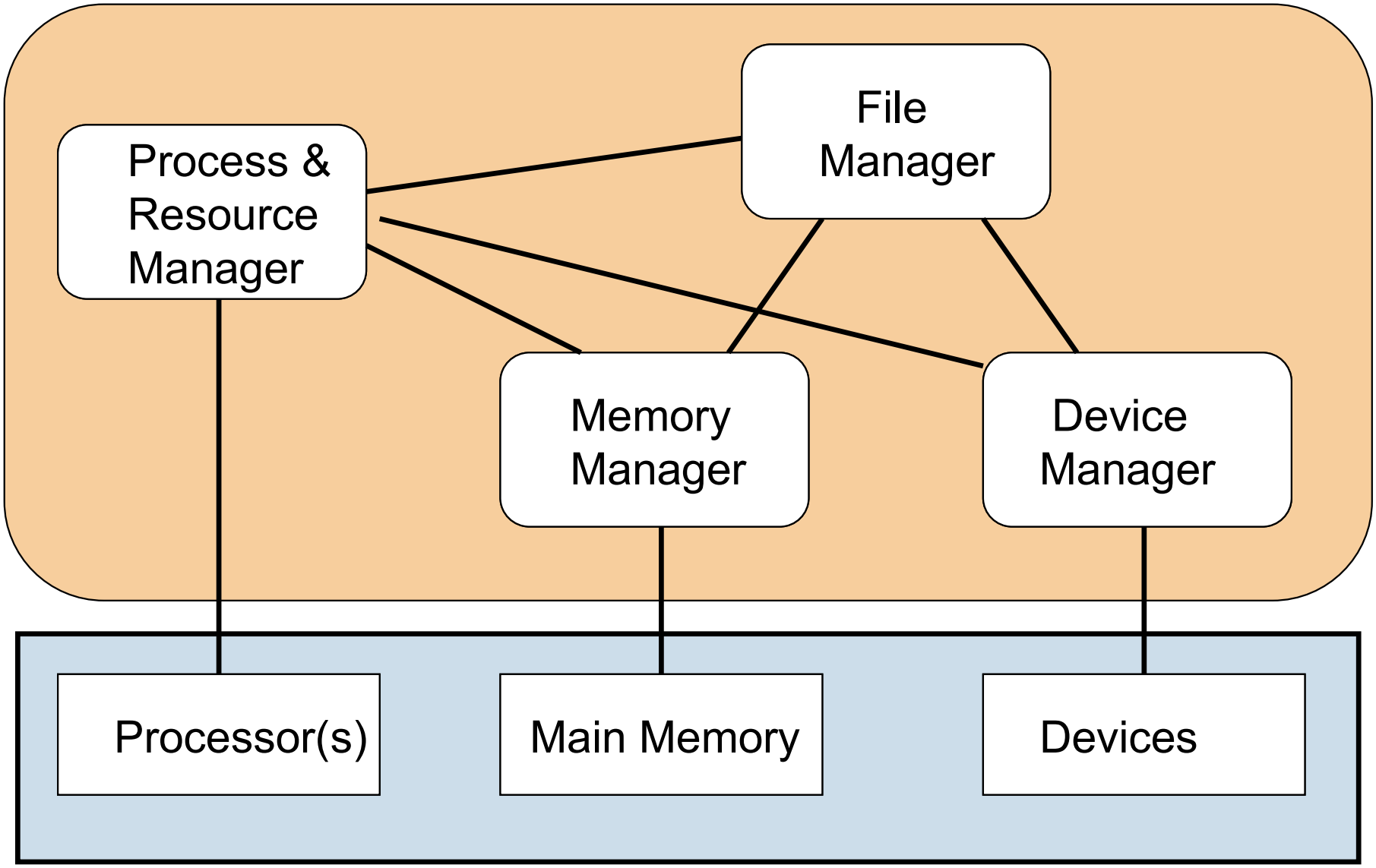
- Program development utility programs
  - Editors, debuggers, frameworks, ..
- Program execution - steps
  - Loading instructions and data in memory
  - Initialization of I/O devices and files
  - Scheduling of resources
- Access to I/O devices
  - Provides Uniform interface, hides details
- Controlled access to files
  - Understanding the structure of the data contained in files
  - Authorization, sharing, protection, caching

Services cont…

- System access
  - Protection, authorization, resolve conflicts
- Error detection and response
  - Hardware errors: memory error or device failure
  - Software errors: arithmetic errors, access forbidden memory locations, allocation errors

  OS clears the error and responds: report or end program
- Accounting
  - Statistics to monitor performance
  - used to anticipate future enhancements and tuning
  - collect statistics (billing) - multiuser

Basic OS Organization

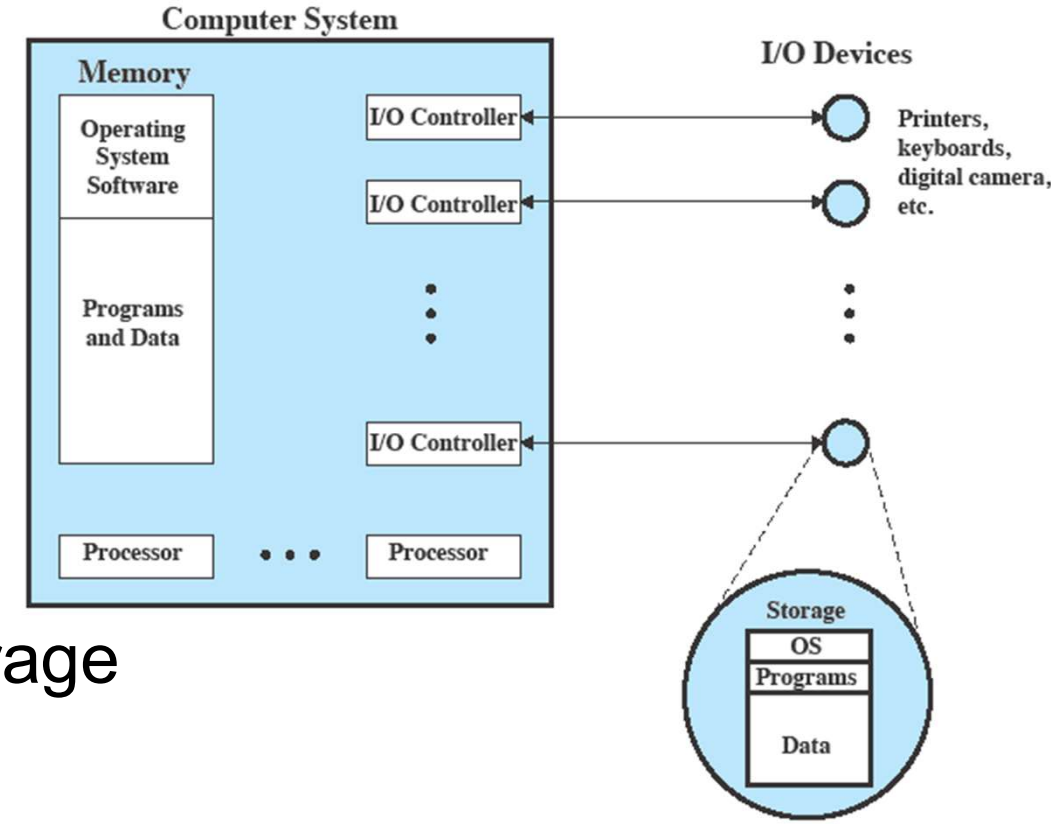


OS as a Resource Manager

- OS is a set of computer programs
  - executes as ordinary computer software
- The key difference is in the intent
  - OS *directs* the processor in the use of resources
  - *Relinquishes* control of the processor to execute other programs
- Built around kernel or nucleus
  - Portion of OS that is in main memory
  - Contains most-frequently used functions

OS as a Resource Manager

- Resources are managed by OS:
  - I/O devices
  - Memory
  - Processors
- Portion of OS is in main memory, kernel, rest on storage
- Main memory contains also user programs and data
- Allocation of main memory is controlled by OS and memory management hardware



Modes of Operation

- Considerations of memory protection and privileged instructions lead to the concept of modes of operation
- **Kernel**
  - Portion of operating system that is in main memory
  - Contains most-frequently used functions
  - Also called the ***nucleus***
- **User Mode**
  - User program executes in user mode
  - Certain areas of memory protected from user access
  - Certain instructions may not be executed
- **Kernel Mode**
  - Monitor executes in kernel mode
  - Privileged instructions may be executed, all memory accessible.

Multiprogramming

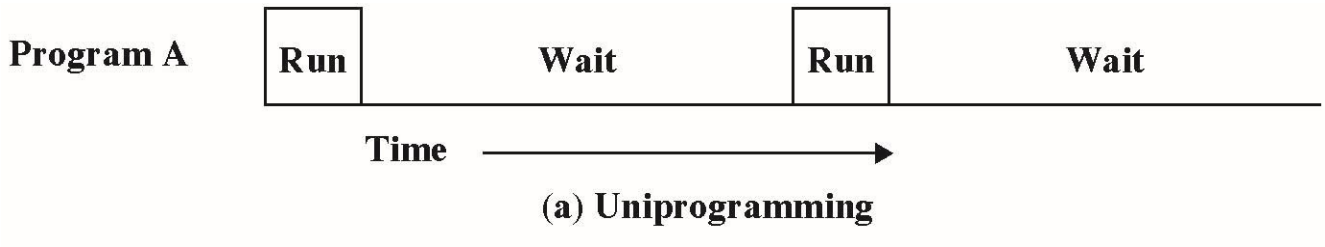
Read one record from file	15 μs
Execute 100 instructions	1 μs
Write one record to file	15 μs
TOTAL	31 μs

Percent CPU Utilization $= \frac{1}{31} = 0.032 = 3.2\%$

- CPU is often idle
  - I/O devices are slow compared to processor
- Multiprogramming is a technique to execute many programs simultaneously with a single processor
- The processes all reside in main memory at same time
- The OS picks and begins to executes one of the jobs in the main memory
- The job executes until it completes or “blocks” on an I/O request at which time the CPU switches to another job
- CPU is not idle, as long as there are requests

Uniprogramming

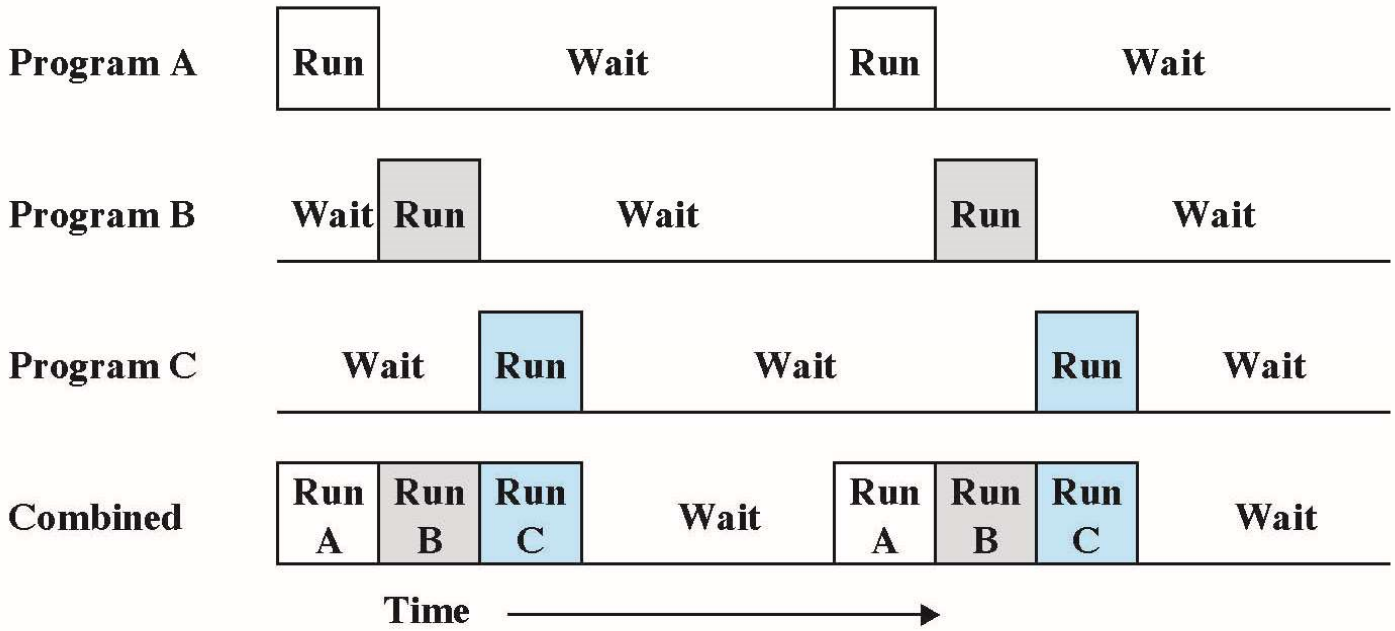
- Processor must wait for I/O instruction to complete before proceeding



(a) Uniprogramming

Multiprogramming

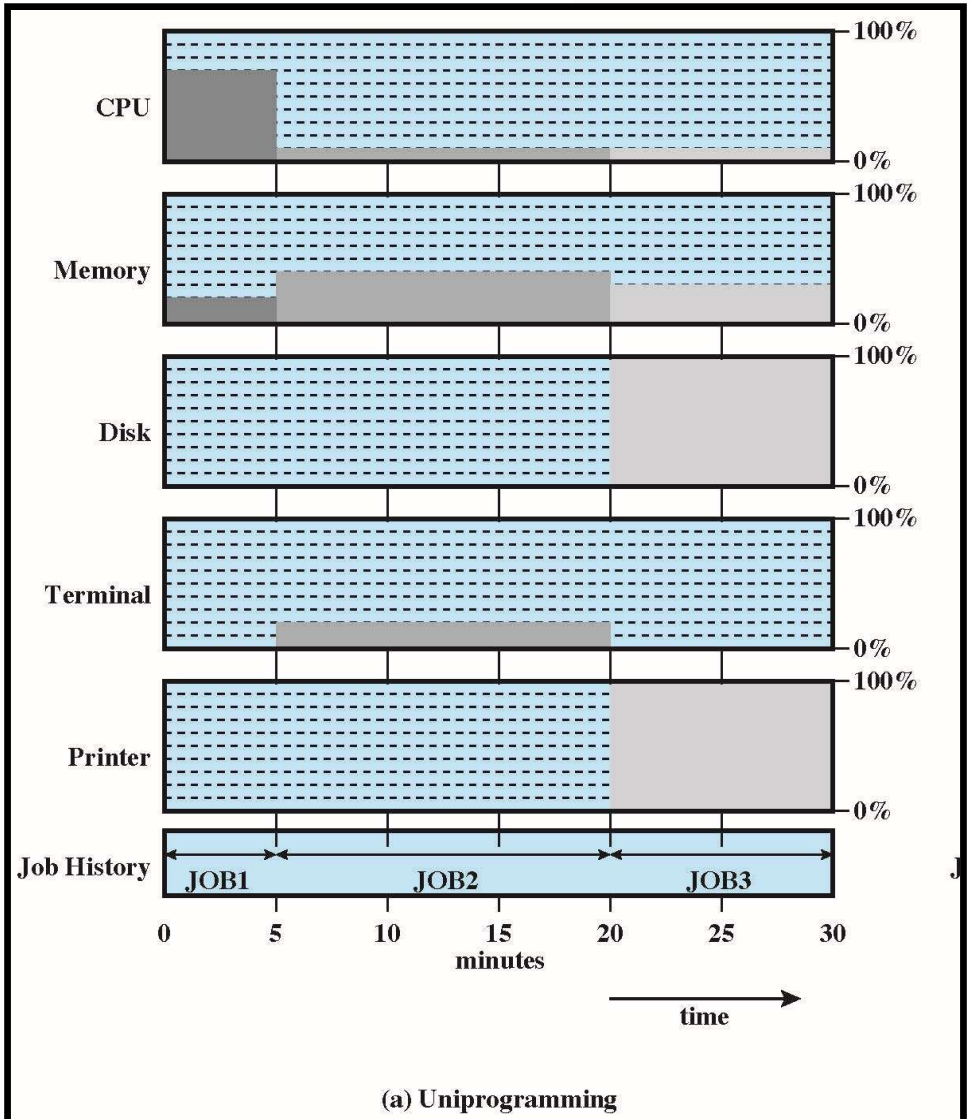
- When one job needs to wait for I/O, the processor can switch to another job



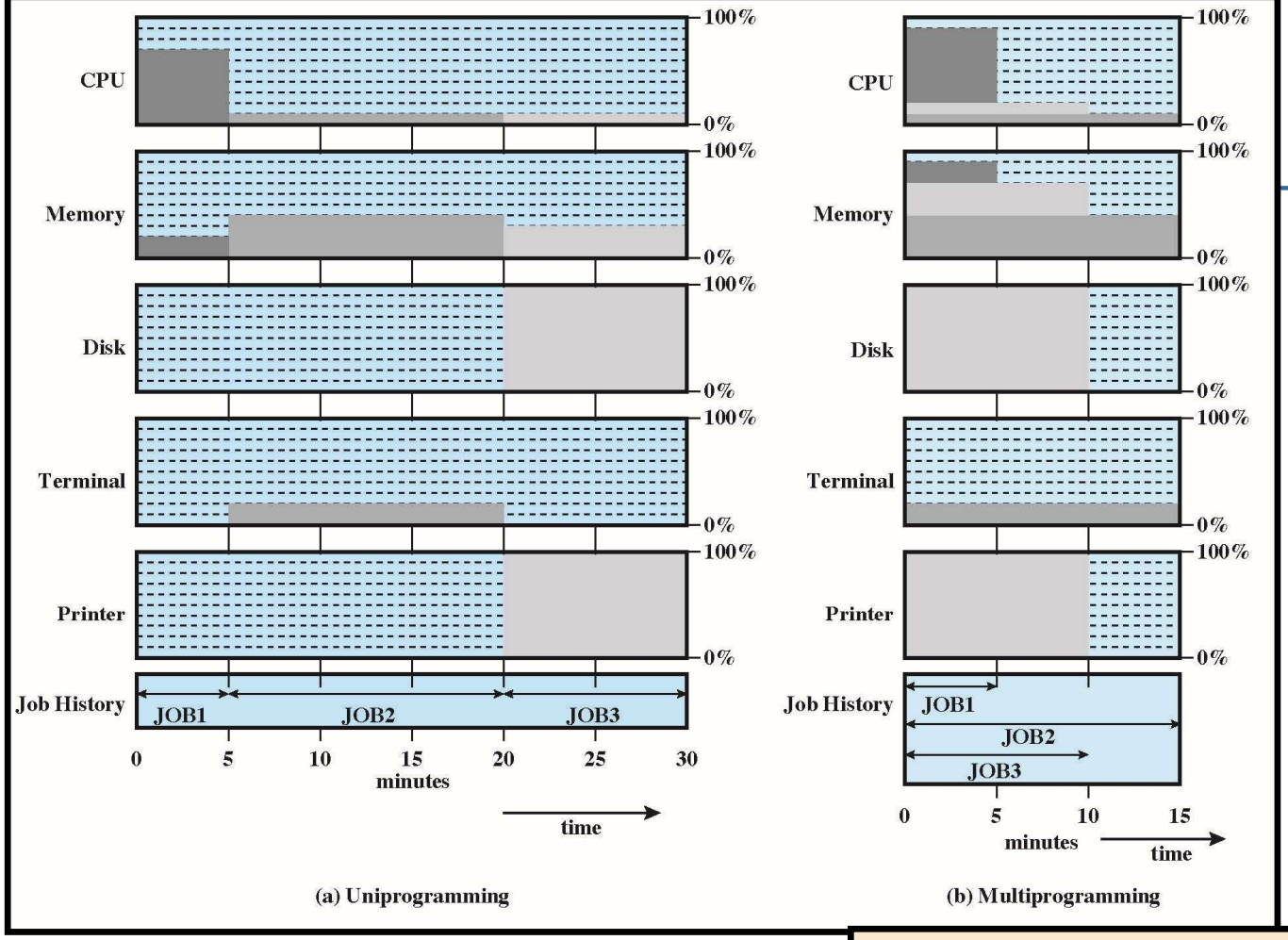
(c) Multiprogramming with three programs

Example

	JOB1	JOB2	JOB3
Type of job	Heavy compute	Heavy I/O	Heavy I/O
Duration	5 min	15 min	10 min
Memory required	50 M	100 M	75 M
Need disk?	No	No	Yes
Need terminal?	No	Yes	No
Need printer?	No	No	Yes



(a) Uniprogramming



	JOB1	JOB2	JOB3
Type of job	Heavy compute	Heavy I/O	Heavy I/O
Duration	5 min	15 min	10 min
Memory required	50 M	100 M	75 M
Need disk?	No	No	Yes
Need terminal?	No	Yes	No
Need printer?	No	No	Yes

	Uniprogramming	Multiprogramming
Processor use	22%	43%
Memory use	30%	67%
Disk use	33%	67%
Printer use	33%	67%
Elapsed time	30 min.	15 min.
Throughput rate	6 jobs/hr	12 jobs/hr
Avg response time	18 min.	10 min.

Time Sharing

- Extension to multiprogramming to handle multiple interactive jobs
- **Processor** time **shared** between users: Multiple jobs are executed simultaneously by the CPU switching between tasks according to some time slice or Quantum
- Every time slice: current user is preempted and another user loaded in
- Allow several users to interact at the same time – transaction processing
- Users feel of *virtual ownership* of the computer
- Emphasizes response time over processor use
- Compatible Time Sharing System (CTSS) – 1962
  - 32K 36-bit words, switched users every 0.2 sec, Support up to 32 users

	Batch Multiprogramming	**Time Sharing**
Principal objective	Maximize processor use	Minimize response time
Source of directives to operating system	Job control language commands provided with the job	Commands entered at the terminal

Batch Multiprogramming vs. Time Sharing

Problems and Issues with Time Sharing

- Multiple jobs in memory must be protected from interfering with each other's data
- File system must be protected so that only authorised users can access
- Contention for resources must be handled
  - Printers, storage etc

Major Achievements

- Operating Systems are among the most complex pieces of software ever developed
- Major advances in development include:
  - Processes
  - Memory management
  - Information protection and security
  - Scheduling and resource management
  - System structure

Process

- Fundamental to the structure of OS

A ***process*** can be defined as:

a program in execution

an instance of a running program

the entity that can be assigned to, and executed on, a processor

a unit of activity characterized by a single sequential thread of execution, a current state, and an associated set of system resources

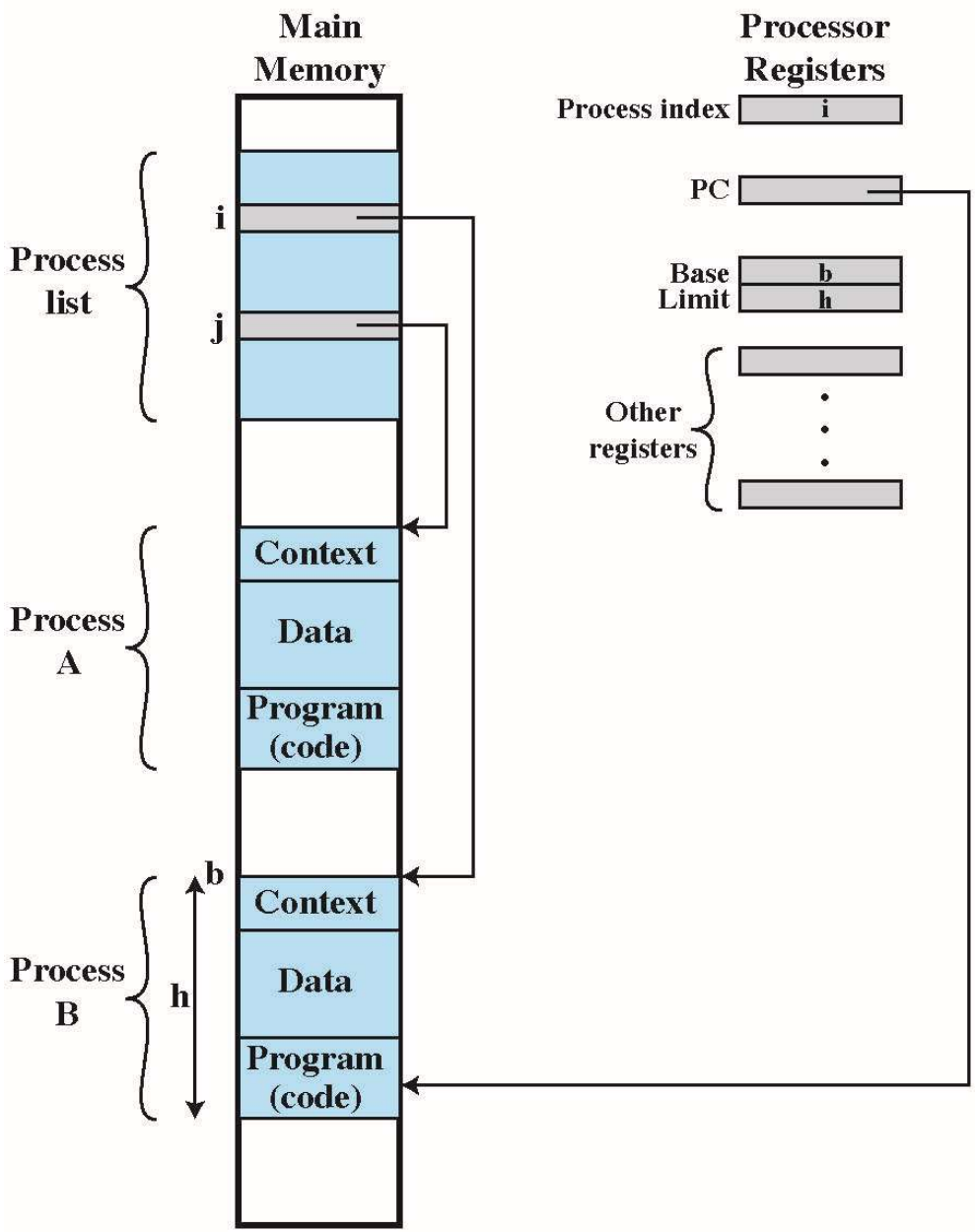
Processes

- Three major lines of computer system development created problems in timing and synchronization that contributed to the development:
  - Multiprogramming batch operations: processor is switched among the various programs residing in main memory – maximize resource efficiency
  - Time sharing be responsive to the individual user but be able to support many users simultaneously
  - Real-time transaction processing systems a number of users are entering queries or updates against a database
- Principle tool available to system programmers in developing multiprogramming and time sharing systems was the interrupt!
  - Process is suspended due to a defined event, e.g. I/O completion
  - CPU has to save context (PC, registers, etc..)
  - CPU branch to interrupt handler routine, process the interrupt
  - CPU resume processing another job

Processes

- Coordination of processes turned out difficult due to errors caused by
  - Improper synchronization
    - Improper design of the signaling mechanism can result in signals being lost or duplicate signals being received
  - Failed mutual exclusion
    - attempt to make use of a shared resource at the same time e.g. two users trying to edit same file - OS has to permit only one
  - Non-determinate program operation
    - Programs may overwrite common memory area since they share memory and their execution is interleaved by the processor
  - Deadlocks
    - Two or more programs can hang waiting for each other
- Systematic way to of coordination among programs executing on the processor is essential

Process

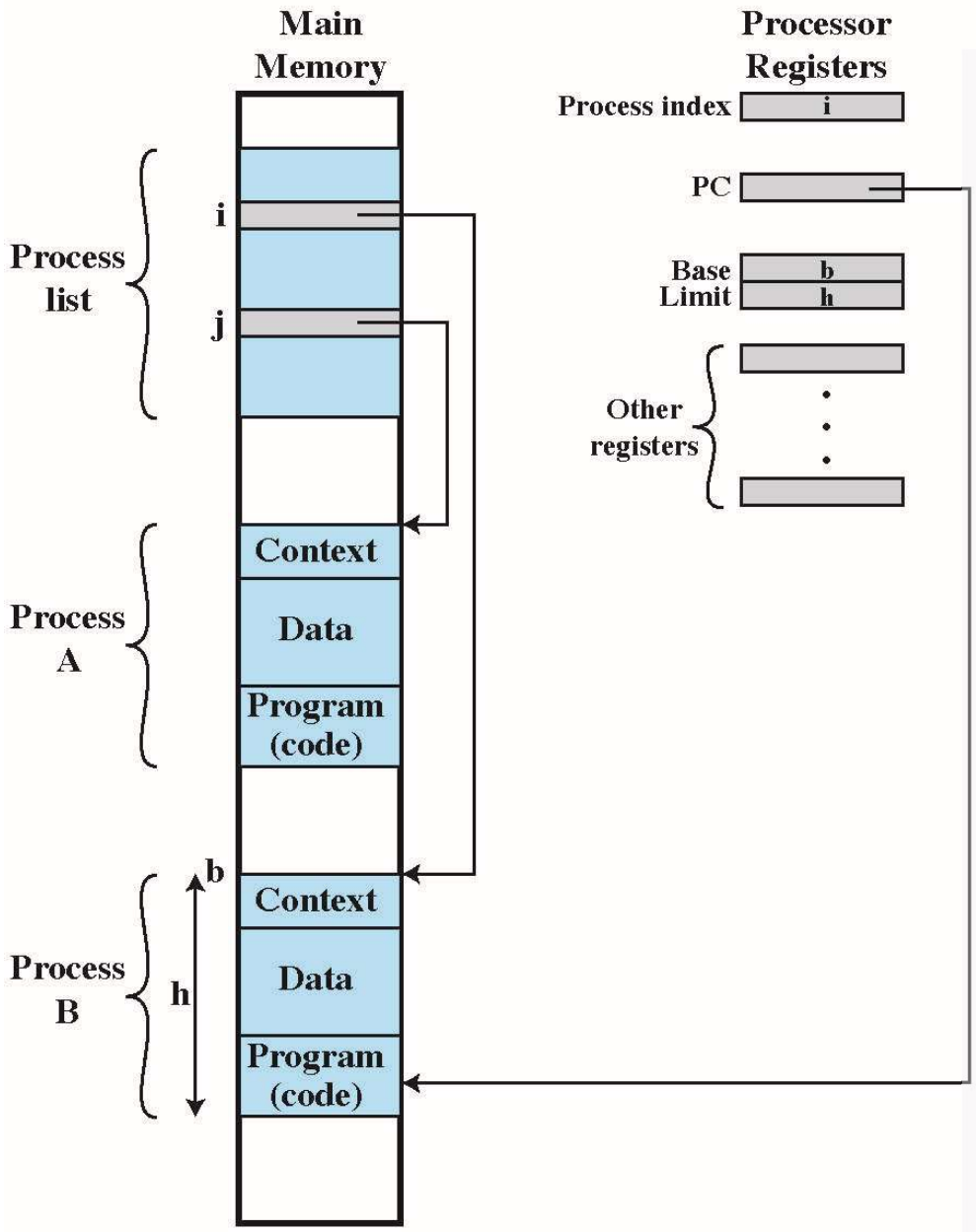


Consists of three components

1. executable program
2. associated data (variables, workspace, buffers, stacks)
3. execution context: internal data used by OS to be able to control the process e.g. PC, registers, priority

- The execution context/ Process State is essential:
  - it is the internal data by which the OS is able to supervise and control the process
  - includes the contents of the various process registers
  - includes information such as the priority of the process and whether the process is waiting for the completion of a particular I/O event

Process



Example

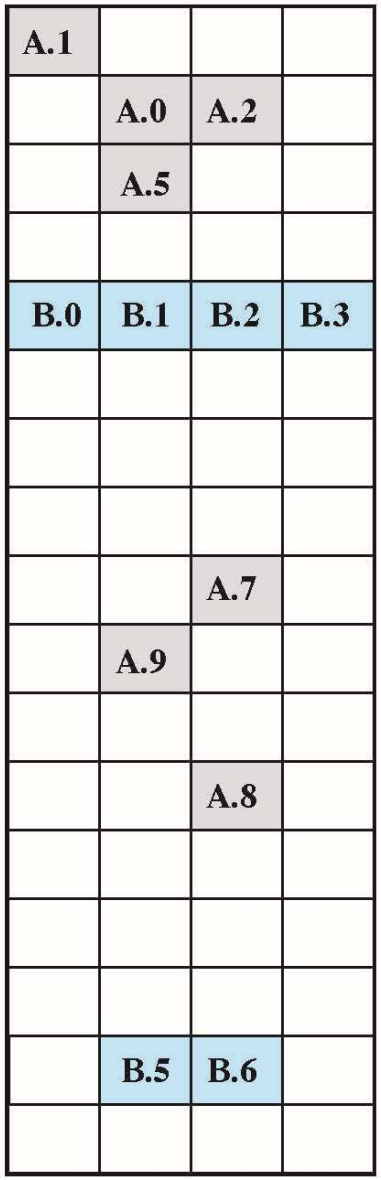
- Two processes A, B exists in main memory, B is executing
- Each recorded in process list
- Process index register points to index of current running process
- Program counter: points to next instruction
- Base and limit register: define region in memory occupied by process
- Later OS perform context switch to A
- Context: contains ***entire state*** of a process

Memory Management

- Five principal storage management responsibilities
  1. Process isolation prevents interference among programs; data + instruction
  2. Automatic allocation and management transparent to the user
  3. Support for modular programming (e.g. dynamic process creation and elimination)
  4. Protection and access control for sharing
  5. Long-term storage
- These requirements typically met by
  - Virtual memory: allows programs to address memory from a logical point of view without regard to the limits of physical memory
  - File system facilities: implements long-term store with named objects: files

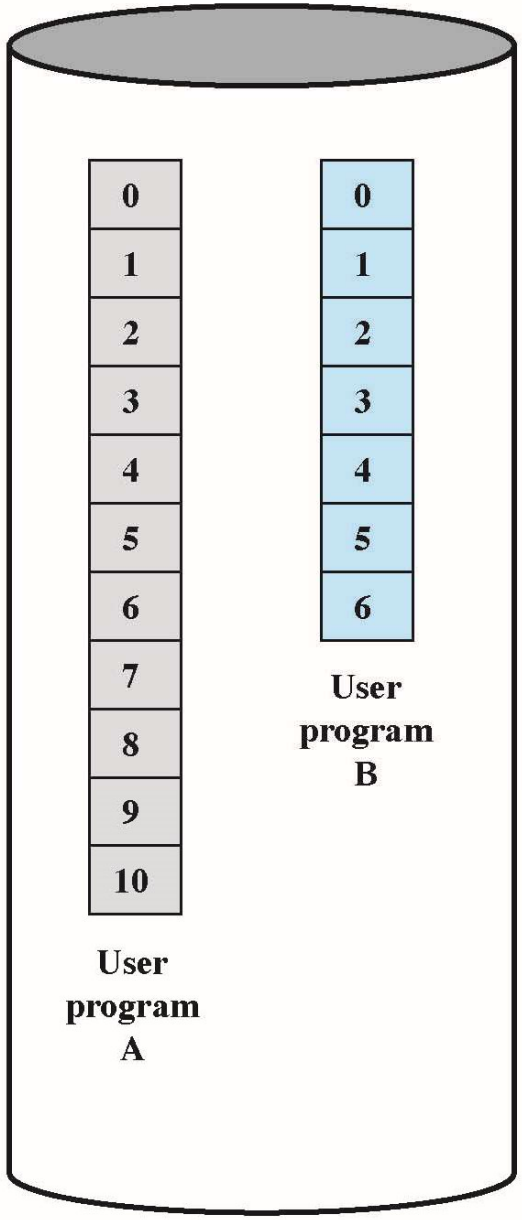
Paging

- Process is divided into a number of fixed-size blocks, called **pages**
- **Virtual address** is a page number and an offset within the page
- MM provides **dynamic mapping** between virtual and real address
- Each page may be located anywhere in main memory
- If reference made to a page not in main memory, MM detects this and load it from disk



Main Memory

Main memory consists of a number of fixed-length frames, each equal to the size of a page. For a program to execute, some or all of its pages must be in main memory.



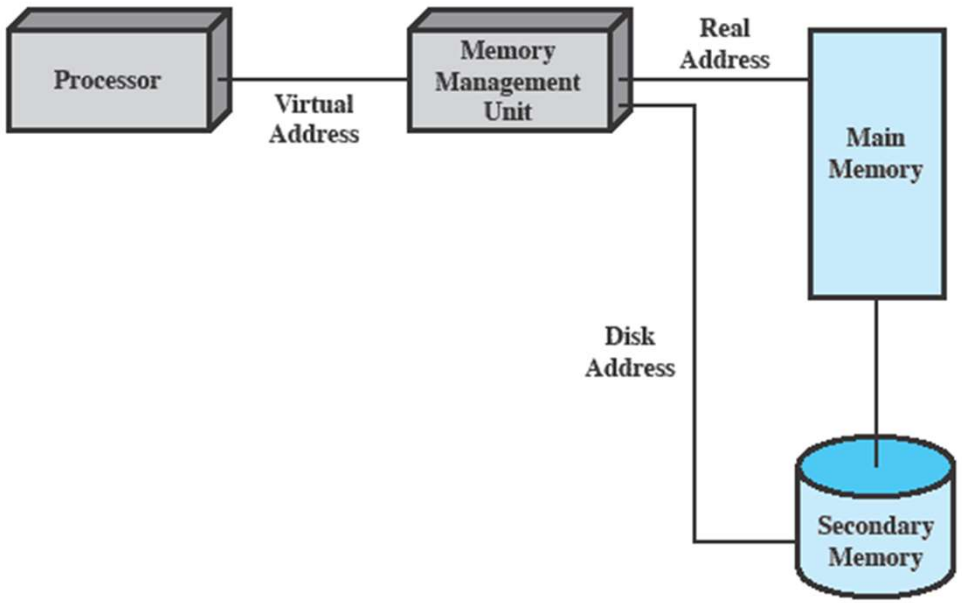
Disk

Secondary memory (disk) can hold many fixed-length pages. A user program consists of some number of pages. Pages for all programs plus the operating system are on disk, as are files.

Virtual Memory Addressing

Address translation hardware (Memory Management Unit) is interposed between the processor and memory

- Programs reference locations using virtual addresses, which are mapped into real main memory addresses.
- If a reference is made to a virtual address not in real memory, then a the desired block of data is brought in from auxiliary storage into main memory
- During this activity, the process that generated the address reference must be suspended.



The OS designer needs to develop

- an address translation mechanism that generates little overhead
- storage allocation policy that minimizes the traffic between memory levels.

Data Protection and Security

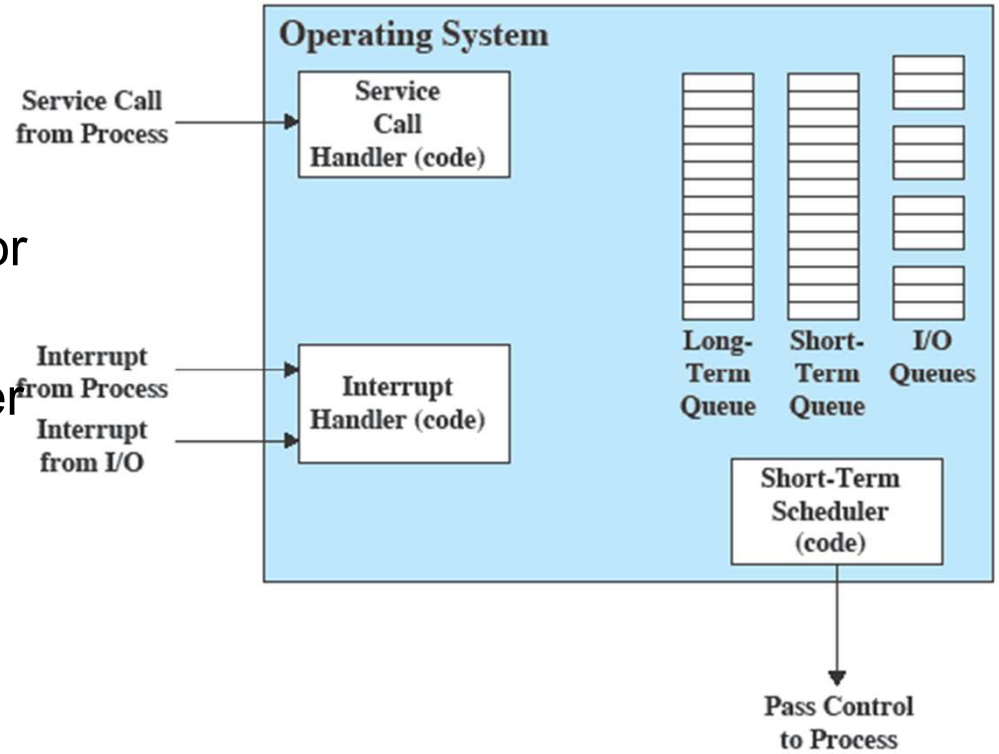
- Availability
  - Concerned with protecting the system against interruption
- Confidentiality
  - Assuring that users cannot read data for which access is unauthorized
- Data integrity
  - Protection of data from unauthorized modification
- Authenticity
  - Concerned with the proper verification of the identity of users and the validity of messages or data

Scheduling and Resource Management

- OS has to
  - Manage resources e.g. main memory, CPU, I/O devices
  - Schedule their use among active processes
- Resource allocation and scheduling policy must consider 3 factors
  - Fairness
    - Give equal and fair access to resources
  - Differential responsiveness
    - Discriminate among different classes of jobs
  - Efficiency
    - Maximize throughput, minimize response time, and accommodate as many uses as possible - conflict

Key Elements of OS in Scheduling

- OS maintains 3 types of Queues
  - Short term queue:
    - processes in (or partially) main memory ; ready to run
    - Short term scheduler picks a process (RR, priority)
  - Long term queue:
    - new jobs waiting to use processor
    - OS allocate a portion of main memory to a process and transfer it from long to short term queue
    - OS must not over commit memory or CPU with many requests
  - I/O queue for each I/O device
    - OS must decide which process to assign to an available I/O device
- A process invokes a service by a service call: entry point into OS



System Structure

- Layered architecture - design the OS as a series of levels where each level is performing a related subset of functions
- Each layer provides services to the layer above and relies on the functions provided by the layer below
- A change in one layer should not affect other layers
- Modular design - decomposes a problem into a number of more manageable sub problems
- Ideally, lower levels deal with a far shorter time scale (10^{-9} of a second)

Model of a Hierarchical OS

Level	Name	Objects	Example Operations
7	Virtual Memory	Segments, pages	Read, write, fetch
6	Local secondary store	Blocks of data, device channels	Read, write, allocate, free
5	Primitive processes	Primitive process, semaphores	Suspend, resume, wait, signal
4	Interrupts	Interrupt-handling	Invoke, mask, unmask, retry programs
3	Procedures	Procedures, call stack, display	Mark stack, call, return
2	Instruction Set	Evaluation stack, scalar and array data	Load, store, add, subtract, branch(m/c language)
1	Electronic circuits	Registers, buses, etc.	Clear, transfer, activate

* Shaded area ➔ represents Hardware

Model of a Hierarchical OS

Level	Name	Objects	Example Operations
13	Shell	User programming environment	Statements in shell language
12	User processes	User processes	Quit, kill, suspend, resume
11	Directories	Directories	Create, destroy, attach, detach, search, list
10	Devices	External devices such as printer, displays and keyboards	read, write, Open, close
9	File system	Files	Create, destroy, open, close read, write
8	Communications	Pipes	Create, destroy, open. close, read, write

Developments Leading to Modern OS

- Microkernel architecture more to come …
- Multithreading
  - processes divided into threads that can share data and run simultaneously
- Symmetric multiprocessing
  - multiple processors share same main memory and I/O facilities
  - All processors can perform the same functions
- Distributed OS
  - provides the illusion of a single primary and secondary memory;
  - has a distributed file system
- Object-oriented design
  - Facilitates the process of adding modular extensions to a small kernel
  - enables programmers to customize an OS without disrupting system integrity
  - Eases the development of distributed OS.

Multithreading

Technique in which a process, executing an application, is divided into threads that can run concurrently

Thread

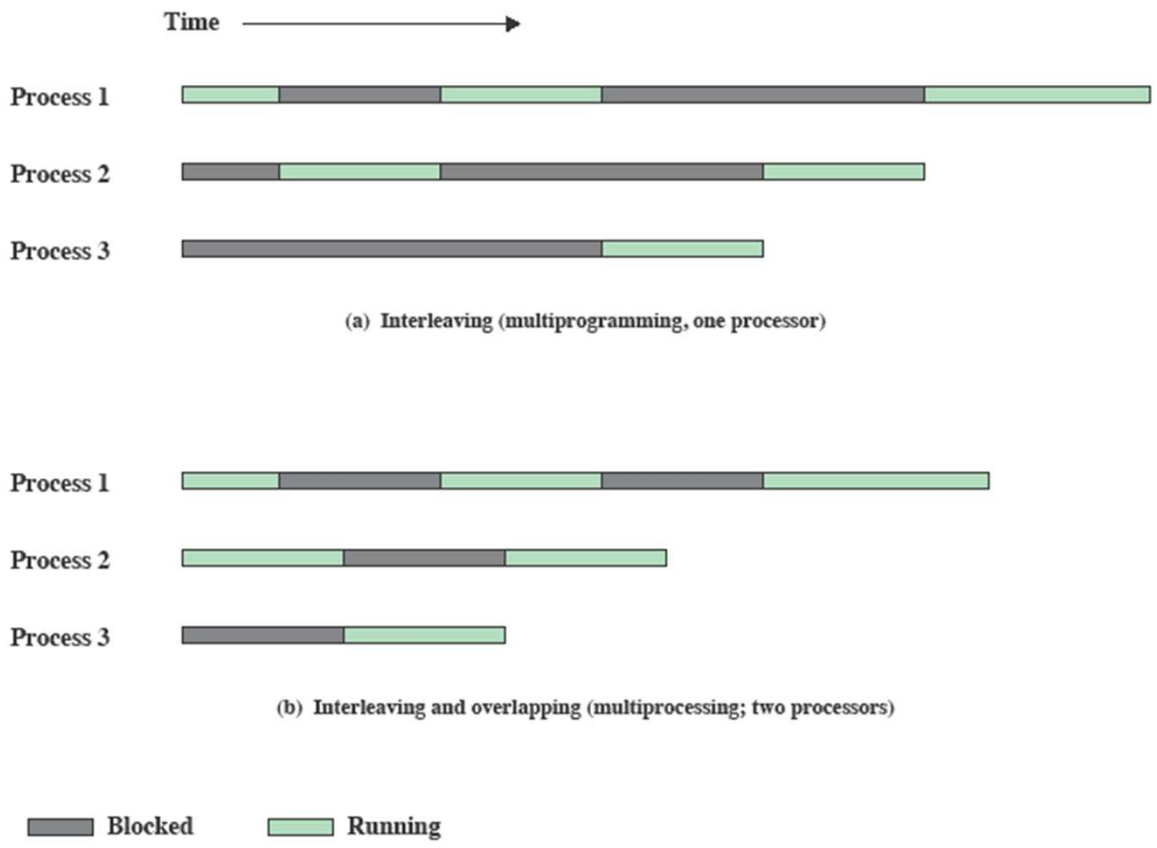
- Dispatchable unit of work
- Includes a processor context (PC and Stack Ptr) and its own data area for a stack
- Executes sequentially and is interruptible

Process

- A collection of one or more threads and associated system resources e.g. memory (code, data), open files and devices
- By breaking a single application into multiple threads, a programmer has greater control over the modularity of the application and the timing of application-related events

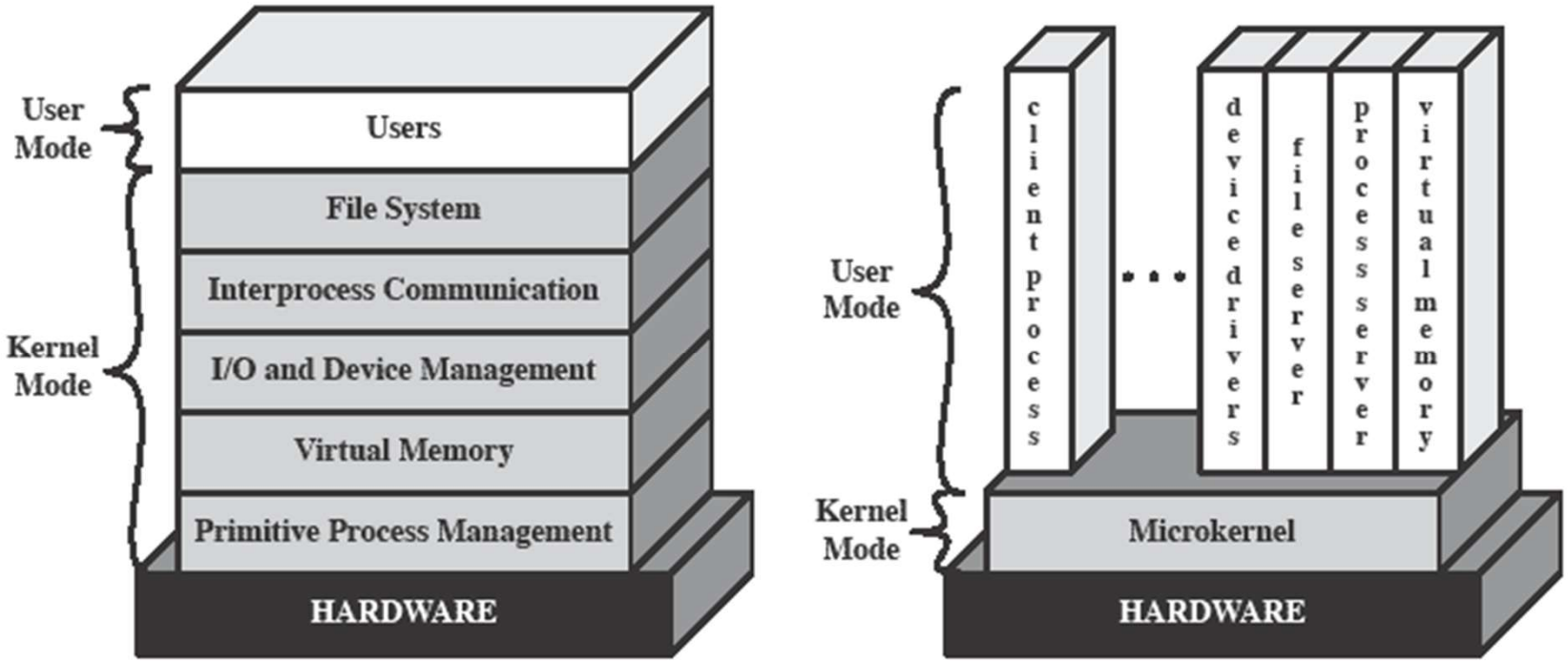
Multiprogramming & Multiprocessing

- Multiprogramming: only one process can execute at a time; meanwhile all other processes are waiting for the processor
- Multiprocessing: more than one process can be running simultaneously, each on a different processor.
- Multithreading and SMP are independent:
  - Multithreading is useful, even on a uniprocessor
  - SMP is useful even for non-threaded processes
  - They complement each other



Microkernel Architecture

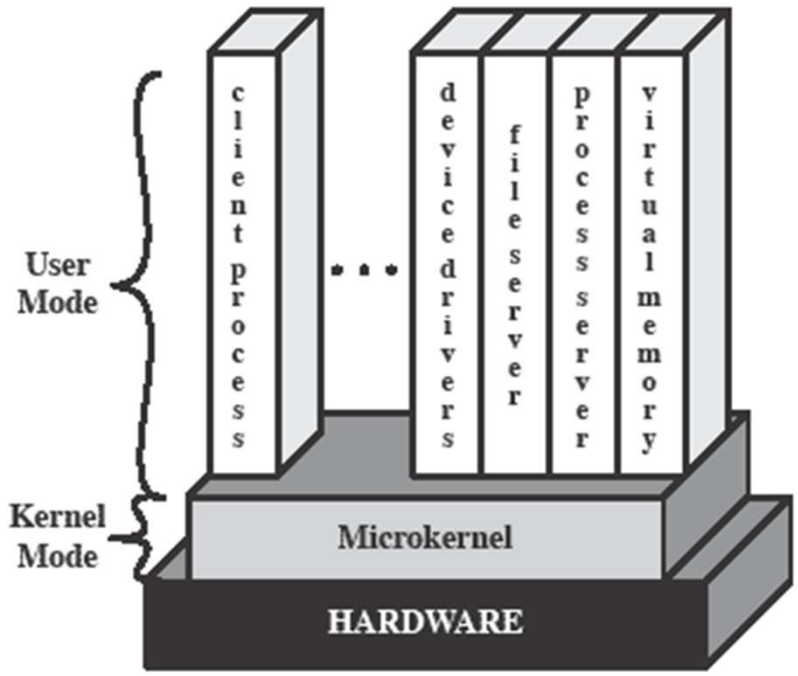
- Monolithic kernel
  - large kernel including scheduling, file system, networking, device drivers, memory management
  - single process with all elements sharing the same address space
- Modern OS use microkernel approach



(a) Layered kernel (b) Microkernel

Microkernels

- Kernel includes only essential core functions (IPC, basic scheduling, ….)
- Other OS services are provided by processes, called servers, run in user mode and treated like any other application by the microkernel
  - Device drivers
  - File systems
  - Virtual memory manager
  - Windowing system
  - Security services
- This approach decouples kernel and server development
- Servers may be customized to specific application or environment requirements



(b) Microkernel

Microkernel Benefits

- Uniform interface on request made by a process
  - Do not distinguish between kernel-level and user-level
  - All services are provided by means of message passing
- Extensibility
  - Allows the addition of new services
- Flexibility
  - New features added, Existing features can be subtracted
- Portability
  - Changes to port the system to a new processor is limited to the microkernel
- Reliability
  - Modular design allows rigorous testing
- Distributed system support
  - A microkernel interacts with local and remote server processes in the same way, facilitating construction of distributed systems.
- Object-oriented operating system

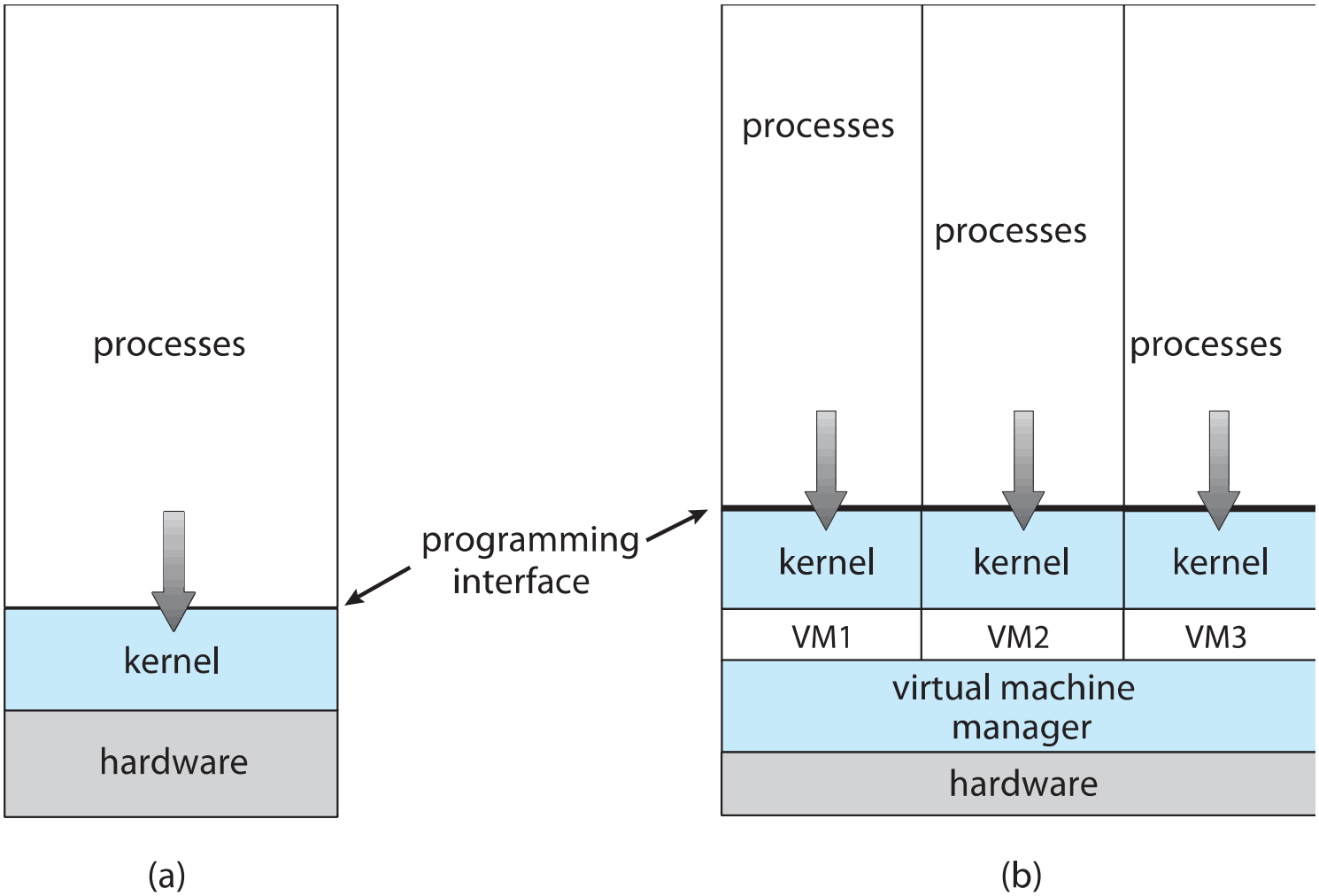
Virtual Machines and Virtualization

- Enables a single PC/server to simultaneously run multiple OSs or multiple sessions of a single OS
- A machine can host numerous applications, including those that run on different operating systems, on a single platform
- A host operating system can support a number of virtual machines (VM)
  - Each has the characteristics of a particular OS and,
  - The characteristics of a particular hardware platform
- Commercial VM e.g. Vmware & Microsoft are widely used
- VM technologies are used in
  - Server environments
  - desktop environments to run multiple operating systems, typically Windows, Linux and Mac OS

Virtual Machines and Virtualization

- Allows operating systems to run applications within other OSes
- Virtualization includes **Emulation** used when source CPU type is different from target type (i.e. PowerPC to Intel x86)
  - slowest method: Every native m/c instruction must be translated to one or several target instructions
  - When computer language not compiled to native code – **Interpretation**
- **Virtualization** – OS natively compiled for CPU, running **guest** OSes also natively compiled
  - Consider VMware running WinXP guests, each running applications, all on native WinXP **host** OS
  - **VMM** (Virtual Machine Manager) provides virtualization services

Virtualization



Virtualization

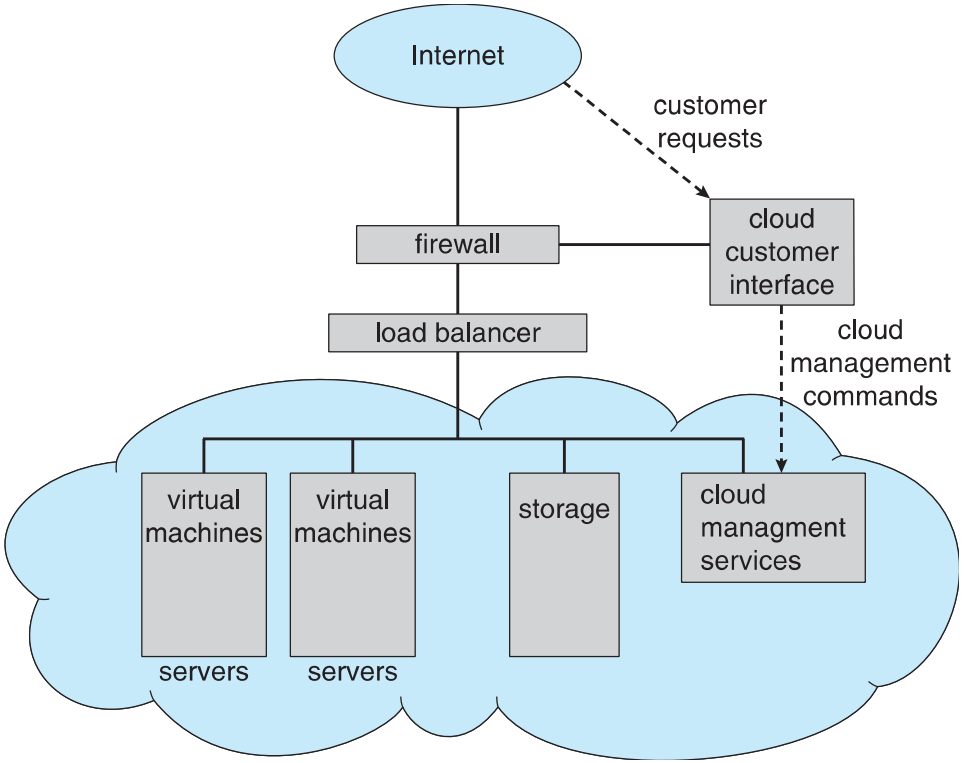
- Use cases involve laptops and desktops running multiple OSes for exploration or compatibility
  - Apple laptop running Mac OS X host, Windows as a guest
  - Developing apps for multiple OSes without having multiple systems
  - QA testing applications without having multiple systems
  - Executing and managing compute environments within data centers
- VMM can run natively, in which case they are also the host
  - There is no general purpose host then (VMware ESX and Citrix XenServer)

Cloud Computing

- Delivers computing, storage, even apps as a service across a network
- Logical extension of virtualization because it uses virtualization as the base for its functionality.
  - Amazon **EC2** has thousands of servers, millions of virtual machines, petabytes of storage available across the Internet, pay based on usage
- Many types
  - **Public cloud** – available via Internet to anyone willing to pay
  - **Private cloud** – run by a company for the company's own use
  - **Hybrid cloud** – includes both public and private cloud components
  - Software as a Service (**SaaS**) – one or more applications available via the Internet (e.g., word processor, payroll)
  - Platform as a Service (**PaaS**) – software stack ready for application use via the Internet (e.g., a database server)
  - Infrastructure as a Service (**IaaS**) – servers or storage available over Internet (e.g., storage available for backup use)

Cloud Computing

- Cloud computing environments composed of
  - traditional Operating systems
  - VMMs managing virtual machines
  - Cloud management tools e.g. Vware, vCloud Director, Eucalyptus
- Internet connectivity requires security like firewalls
- Load balancers spread traffic across multiple applications

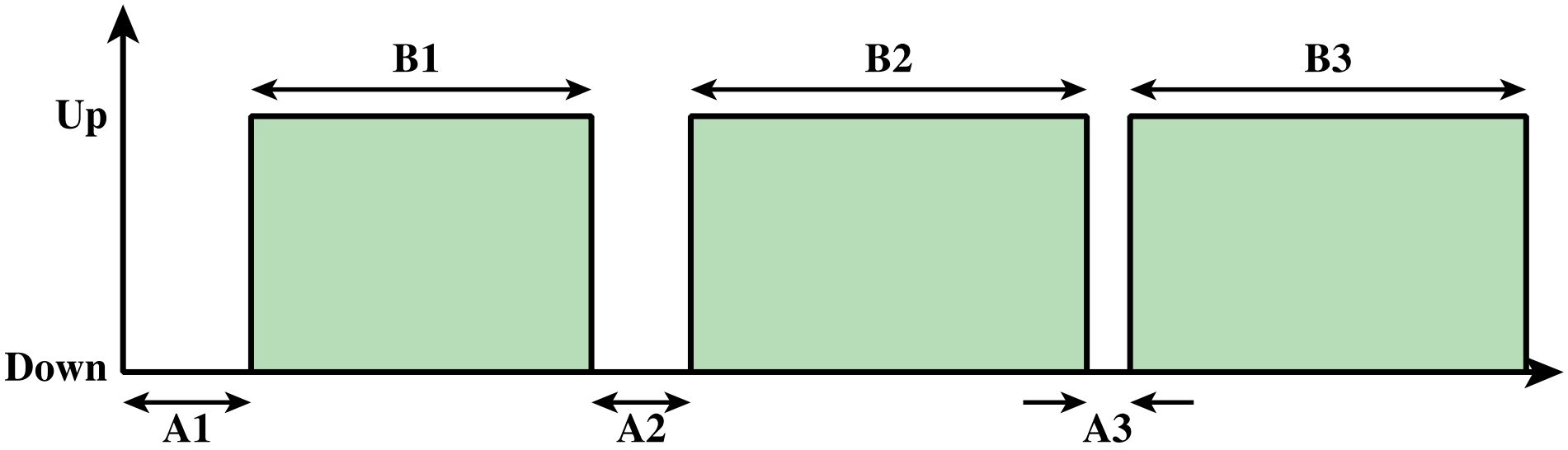


Fault Tolerance

- Refers to the ability of a system or component to continue normal operation despite the presence of hardware or software faults
- Typically involves some degree of redundancy
- Intended to increase the reliability of a system
  - Typically comes with a cost in financial terms or performance
- The extent adoption of fault tolerance measures must be determined by how critical the resource is

Fundamental Concepts

- The basic measures are:
  - **Reliability**
    - *R(t)*
    - Defined as the probability of its correct operation up to time *t* given that the system was operating correctly at time *t=0*
  - **Mean Time To Failure (MTTF)**
    - the average time it takes between failures
  - **Mean Time To Repair (MTTR)**
    - the average time it takes to repair or replace a faulty element
  - **Availability**
    - Defined as the fraction of time the system is available to service users' requests



$$MTTF = \frac{B1 + B2 + B3}{3} \qquad MTTR = \frac{A1 + A2 + A3}{3}$$

Class	Availability	Annual Downtime
Continuous	1.0	0
Fault Tolerant	0.99999	5 minutes
Fault Resilient	0.9999	53 minutes
High Availability	0.999	8.3 hours
Normal Availability	0.99 - 0.995	44-87 hours

Table 2.4 Availability Classes

Faults

- Are defined by the IEEE Standards Dictionary as an erroneous hardware or software state resulting from:
  - Component failure
  - Operator error
  - Physical interference from the environment
  - Design error
  - Program error
  - Data structure error
- The standard also states that a fault manifests itself as:
  - A defect in a hardware device or component
  - An incorrect step, process, or data definition in a computer program

Fault Categories

- **Permanent**
  - A fault that, after it occurs, is always present
  - The fault persists until the faulty component is replaced or repaired
- **Temporary**
  - A fault that is not present all the time for all operating conditions
  - Can be classified as
    - » Transient – a fault that occurs only once
    - » Intermittent – a fault that occurs at multiple, unpredictable times

Methods of Redundancy

Spatial (physical) redundancy

Involves the use of multiple components that either perform the same function simultaneously or are configured so that one component is available as a backup in case of the failure of another component

Temporal redundancy

Involves repeating a function or operation when an error is detected

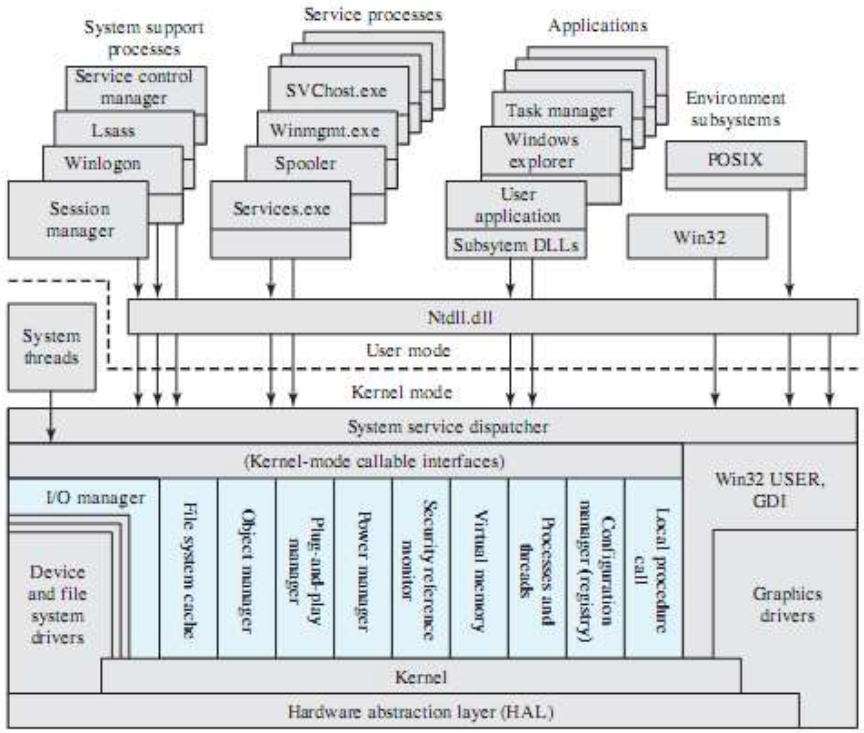
Is effective with temporary faults but not useful for permanent faults

Information redundancy

Provides fault tolerance by replicating or coding data in such a way that bit errors can be both detected and corrected

Windows Architecture

- Modular Structure, Object-oriented design
- **Kernel Mode Components**
  - **Executive** core OS services: Memory management, Process and thread management, Security, I/O, Interprocess communication
  - **Kernel** controls execution of processors. Scheduling, process switching, synch,..
  - **Hardware Abstraction Layer (HAL)** isolates the OS from platform-specific hardware differences
  - **Device Drivers** translate user I/O calls into hardware device I/O requests
  - **Windowing and graphics system**: Implements GUI functions
- Remaining software runs in user mode, access Executive functions thru API



Lsass = local security authentication server
POSIX = portable operating system interface
GDI = graphics device interface
DLL = dynamic link libraries

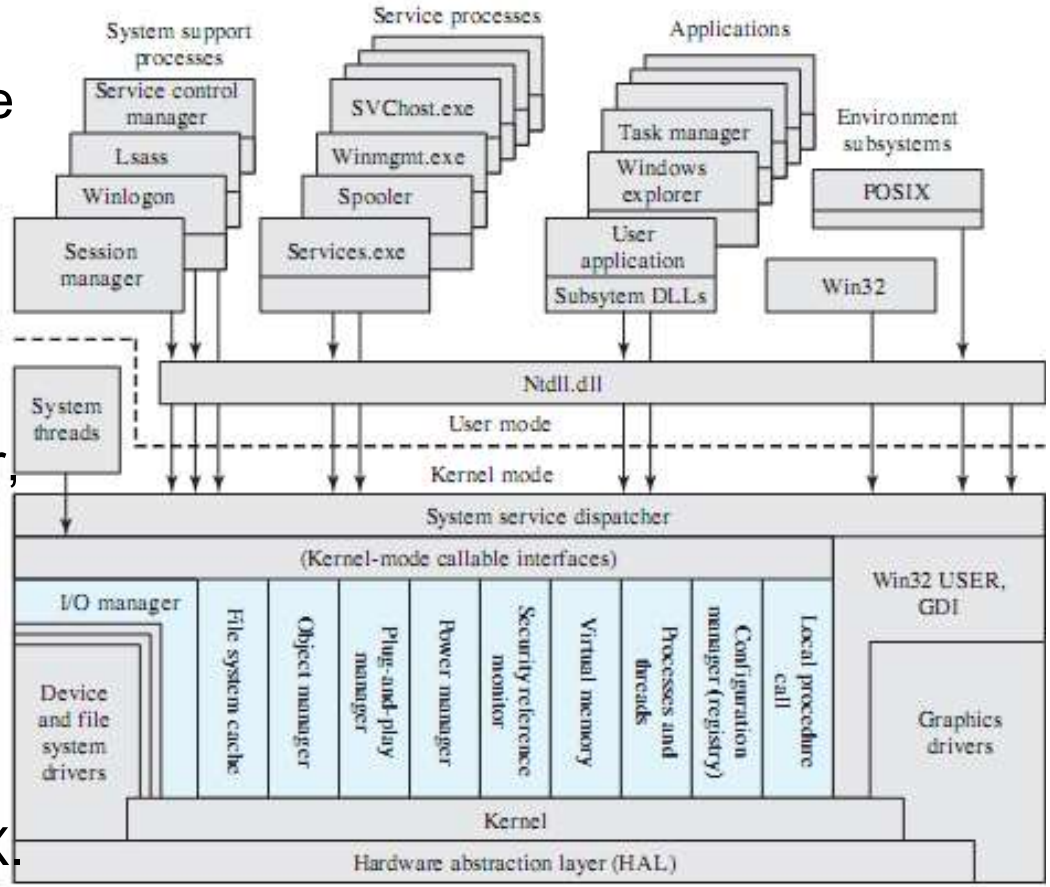
Colored area indicates Executive

Figure 2.13 Windows and Windows Vista Architecture [RUSS05]

Windows Architecture

User-Mode Processes

- **Special system processes**: User mode services needed to manage the system
  - session manager, authentication system, service manager, logon process, ….
- **Service processes**: The printer spooler, the event logger, network services, ….
- **Environment subsystems:**
  - Provide different OS personalities (environments)
  - supported Win32/WinFX and POSIX.
  - DLL convert user application calls to LPC calls and/or native Windows calls
- **User applications**: Executables (EXEs) and DLLs that provide the functionality users run to make use of the system.



Lsass = local security authentication server
POSIX = portable operating system interface
GDI = graphics device interface
DLL = dynamic link libraries

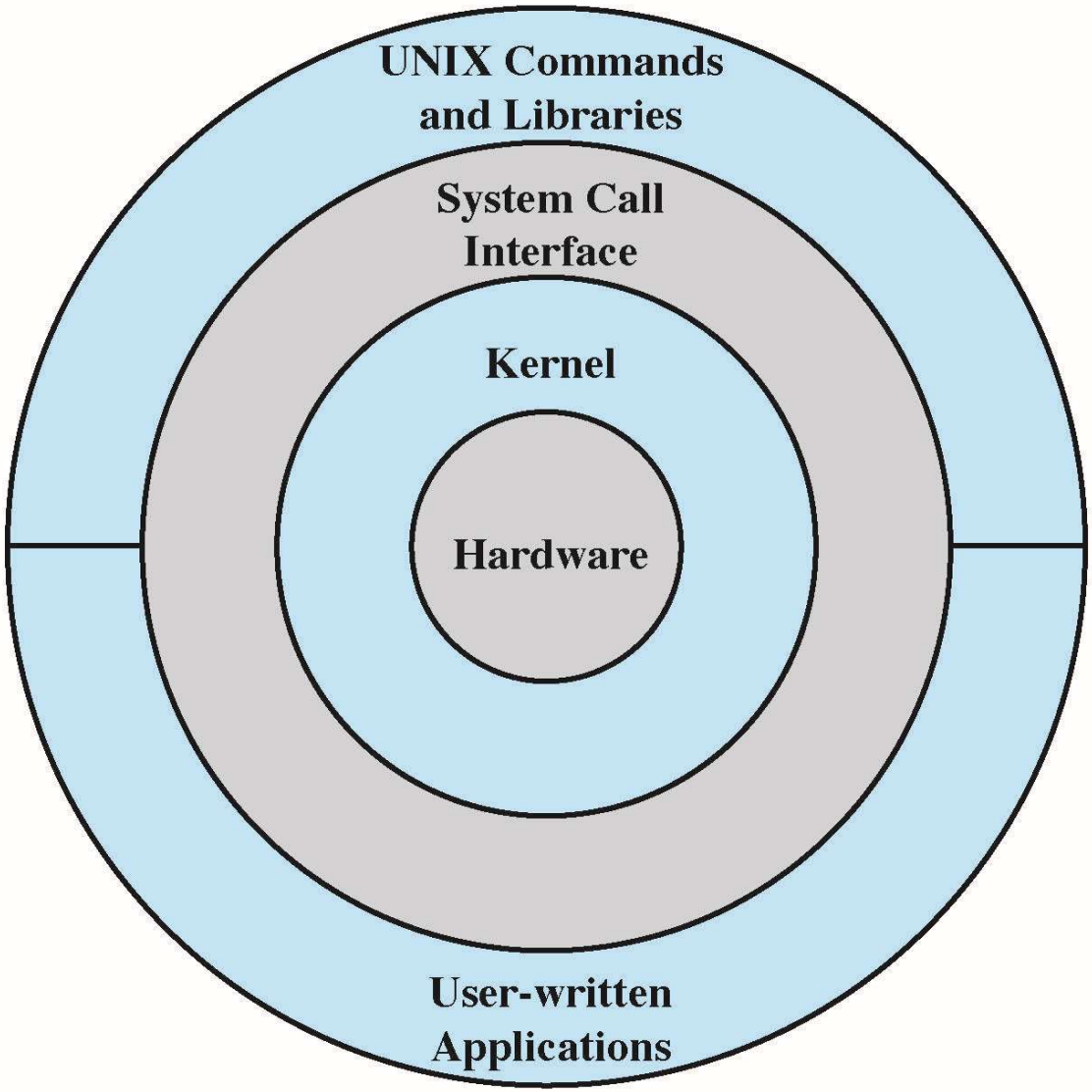
Colored area indicates Executive

Figure 2.13 Windows and Windows Vista Architecture [RUSS05]

Threads and SMP

- Two important characteristics of Windows are its support for threads and for symmetric multiprocessing (SMP)
  - OS routines can run on any available processor, and different routines can execute simultaneously on different processors
  - Windows supports the use of multiple threads of execution within a single process. Multiple threads within the same process may execute on different processors simultaneously
  - Server processes may use multiple threads to process requests from more than one client simultaneously
  - Windows provides mechanisms for sharing data and resources between processes and flexible interprocess communication capabilities

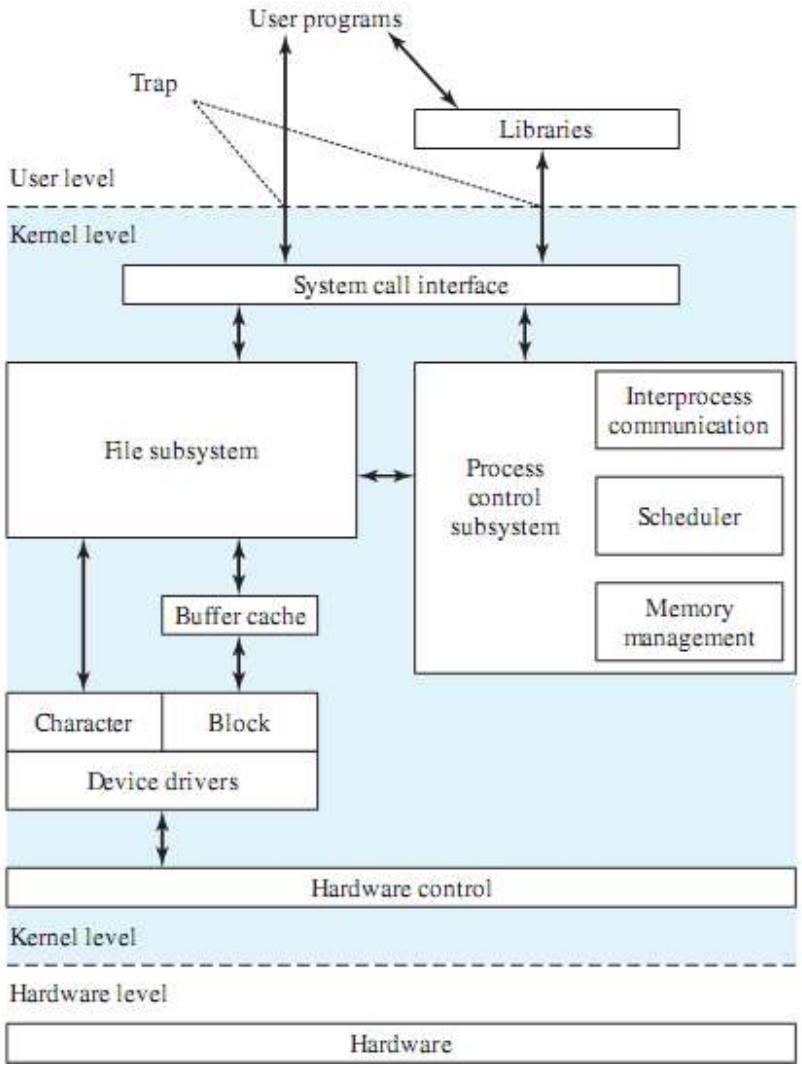
UNIX



- Hardware is surrounded by the OS **kernel**
- Kernel isolates users and applications
- Comes with a number of user services and interfaces
  - Shell
  - C compiler
  - vi editor

Traditional UNIX Kernel

- System call interface is the user boundary to gain access to kernel functions
- OS contains primitive routines that interact directly with the hardware
- Between these two interfaces, the system is divided into two main parts
  - Process control subsystem responsible for memory management, scheduling processes, synchronization and IPC
  - File system
    - exchanges data between memory and external devices using device drivers
    - For block-oriented transfers, a disk cache approach is used



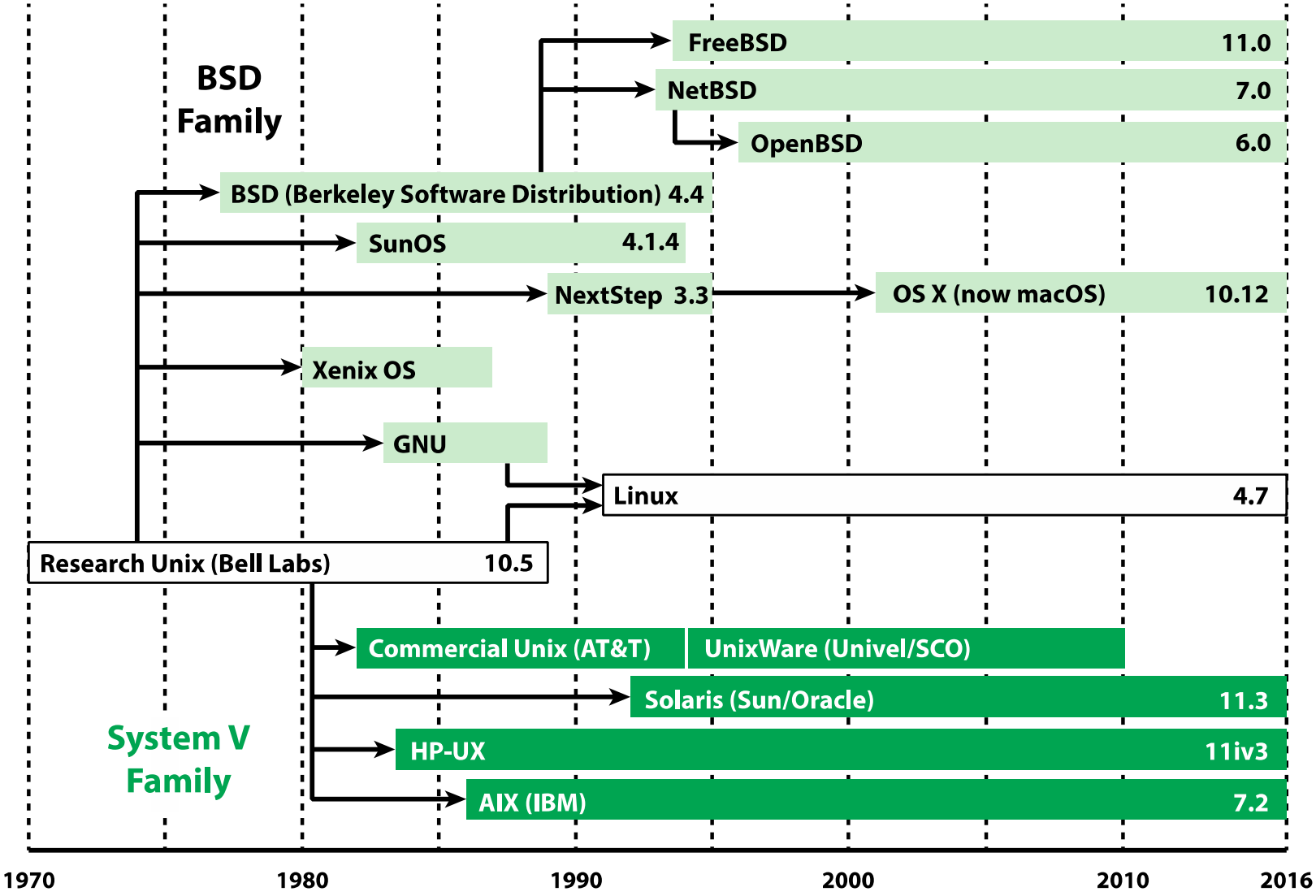


Figure 2.17 Unix Family Tree

System V Release 4 (SVR4)

- Developed jointly by AT&T and Sun Microsystems
- Combines features from SVR3, 4.3BSD, Microsoft Xenix System V, and SunOS
- New features in the release include:
  - Real-time processing support
  - Process scheduling classes
  - Dynamically allocated data structures
  - Virtual memory management
  - Virtual file system
  - Preemptive kernel

System V Release 4 (SVR4)

- Almost a complete rewrite of the System V kernel
- Small core of facilities providing basic functions
- Each of the outer circles represents functions and an interface that may be implemented in a variety of ways.

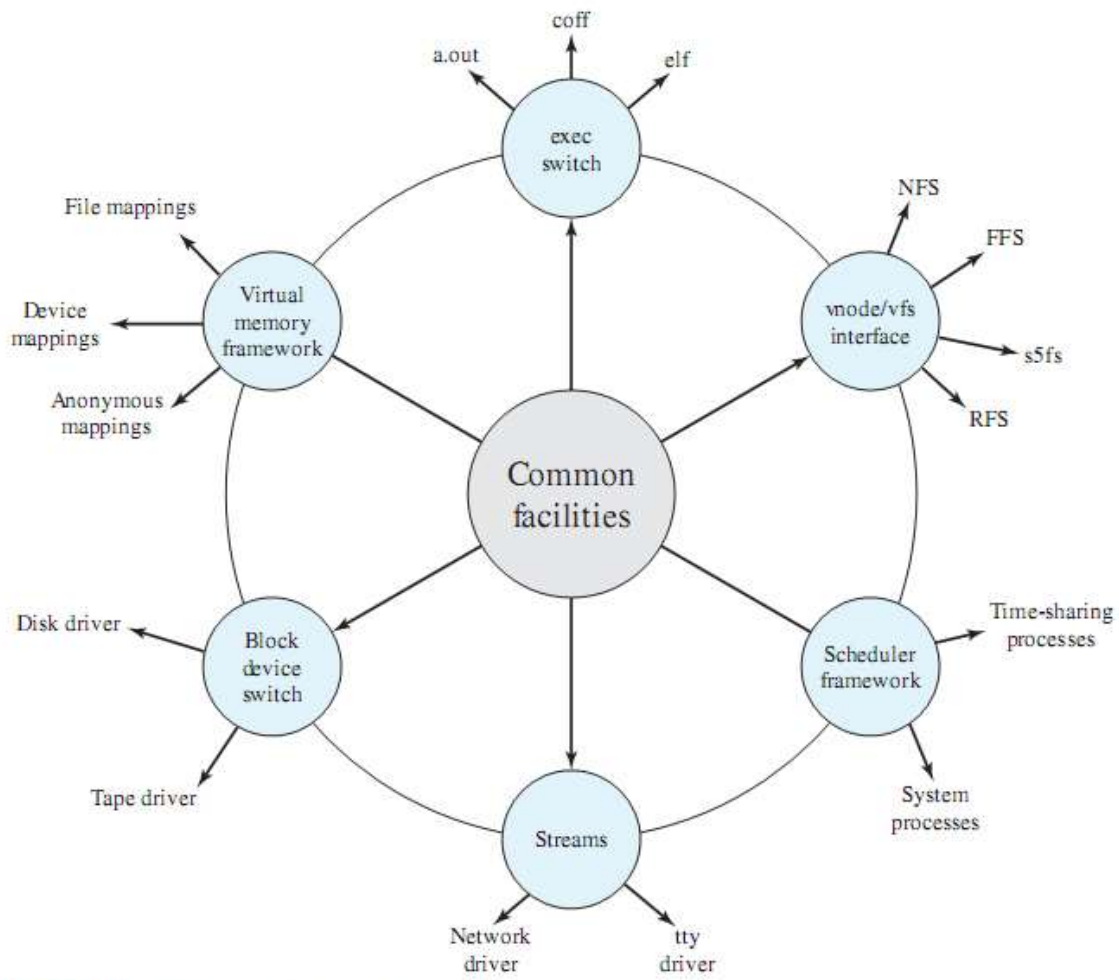


Figure 2.16 Modern UNIX Kernel

BSD

- Berkeley Software Distribution
- 4.xBSD is widely used in academic installations and has served as the basis of a number of commercial UNIX products
- 4.4BSD was the final version of BSD to be released by Berkeley
- There are several widely used, open-source versions of BSD
  - FreeBSD
    - Popular for Internet-based servers and firewalls
    - Used in a number of embedded systems
  - NetBSD
    - Available for many platforms
    - Often used in embedded systems
  - OpenBSD
    - An open-source OS that places special emphasis on security

Solaris 11

- Oracle's SVR4-based UNIX release
- Provides all of the features of SVR4 plus a number of more advanced features such as:
  - A fully preemptable, multithreaded kernel
  - Full support for SMP
  - An object-oriented interface to file systems

LINUX Overview

- Started out as a UNIX variant for the IBM PC
- Linus Torvalds, a Finnish student of computer science, wrote the initial version
- Linux was first posted on the Internet in 1991
- Today it is a full-featured UNIX system that runs on several platforms
- Is free and the source code is available
- Key to success has been the availability of free software packages
- Highly modular and easily configured

Modular Monolithic Kernel

- Includes all OS functionality in one large block running as a single process
- Does not microkernel but achieves advantages due to modular architecture
  - Structured as a collection of modules loaded and unloaded on demand
- Loadable Modules
  - Relatively independent blocks
  - A module is an object file which linked/unlinked from kernel at runtime
  - Have two important characteristics:
    - Dynamic linking:
      - A kernel module can be loaded/ linked while the kernel is executing
      - A module can be unlinked and removed from memory at any time.
      - Saves kernel memory
    - Stackable modules
      - The modules are arranged in a hierarchy with defined dependencies
      - Common code can be moved into a single module
      - Do not unload a module on which other running modules depend

Linux Kernel Components

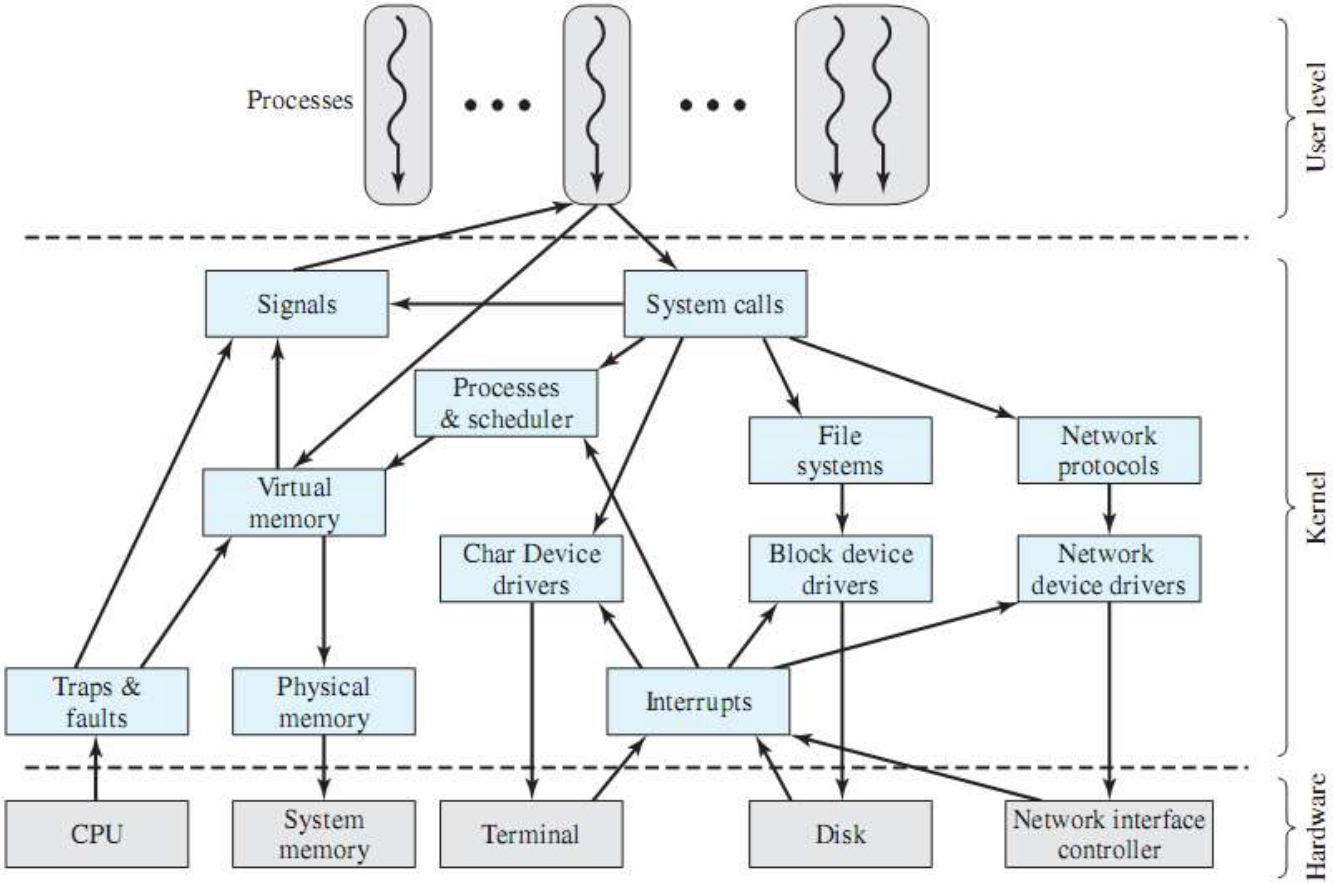


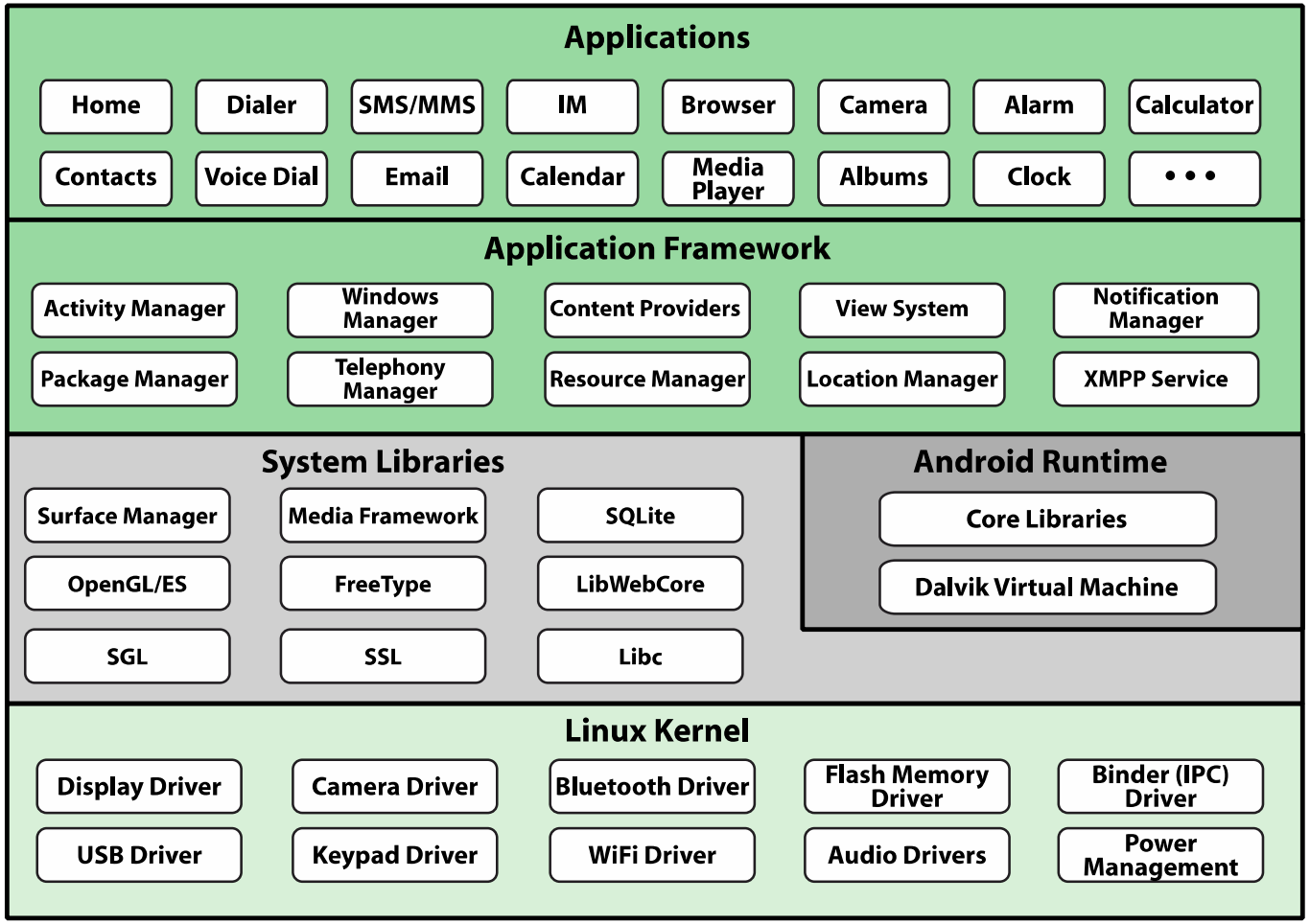
Figure 2.18 Linux Kernel Components

- Several processes / threads run on top of kernel
- Kernel consists of interacting collections of components

Android Operating System

- A Linux-based system originally designed for mobile phones
- The most popular mobile OS
- Development was done by Android Inc., which was bought by Google in 2005
- 1st commercial version (Android 1.0) was released in 2008
- Most recent version 13.0 (Tiramisu) Aug 2022 (12.0 (Snow cone) 11.0, 10.0, Android 9.0 (Pie))
- Android has an active community of developers and enthusiasts who use the Android Open Source Project (AOSP) source code to develop and distribute their own modified versions of the operating system
- The open-source nature of Android has been the key to its success

Android Software Architecture



Implementation:

- Applications, Application Framework: Java
- System Libraries, Android Runtime: C and C++
- Linux Kernel: C

Application Framework

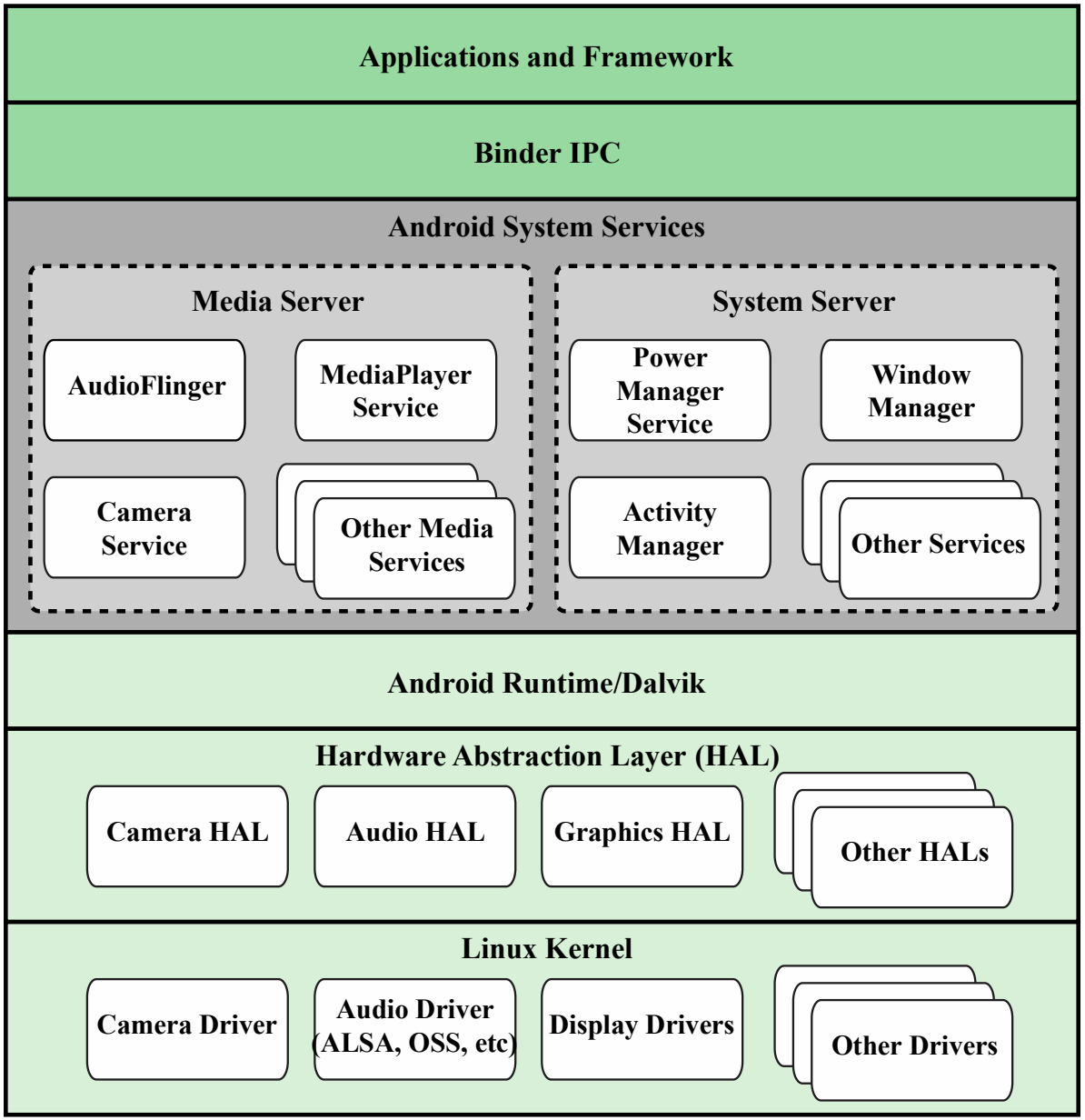
- Provides high-level building blocks accessible through standardized API's that programmers use to create new apps
  - Architecture is designed to simplify the reuse of components
- Key components:

Activity Manager	Window Manager	Package Manager	Telephony Manager
Manages lifecycle of applications	**Java abstraction of the underlying Surface Manager**	**Installs and removes applications**	**Allows interaction with phone, SMS, and MMS services**
Responsible for starting, stopping, and resuming the various applications	**Allows applications to declare their client area and use features like the status bar**		

Application Framework (cont.)

Content Providers	Resource Manager	View System	Location Manager	Notification Manager	XMPP
These functions encapsulate application data that need to be shared between applications such as contacts	Manages application resources, such as localized strings and bitmaps	Provides the user interface (UI) primitives as well as UI Events	Allows developers to tap into location-based services, whether by GPS, cell tower IDs, or local Wi-Fi databases	Manages events, such as arriving messages and appointments	Provides standardized messaging functions between applications

System Libraries

- Collection of useful system functions written in C or C++ and used by various components of the Android system
- Called from the application framework and applications through a Java interface
- Exposed to developers through the Android application framework
- Some of the key system libraries include:
  - Surface Manager
  - OpenGL
  - Media Framework
  - SQL Database
  - Browser Engine
  - Bionic LibC

Android System Architecture